

MODULE -4

Introduction to Server-Side Development with PHP

4.1 What Is Server-side Development?

Server-side development is much more than web hosting: it involves the use of a programming technology like PHP or ASP.NET to create scripts that dynamically generate content.

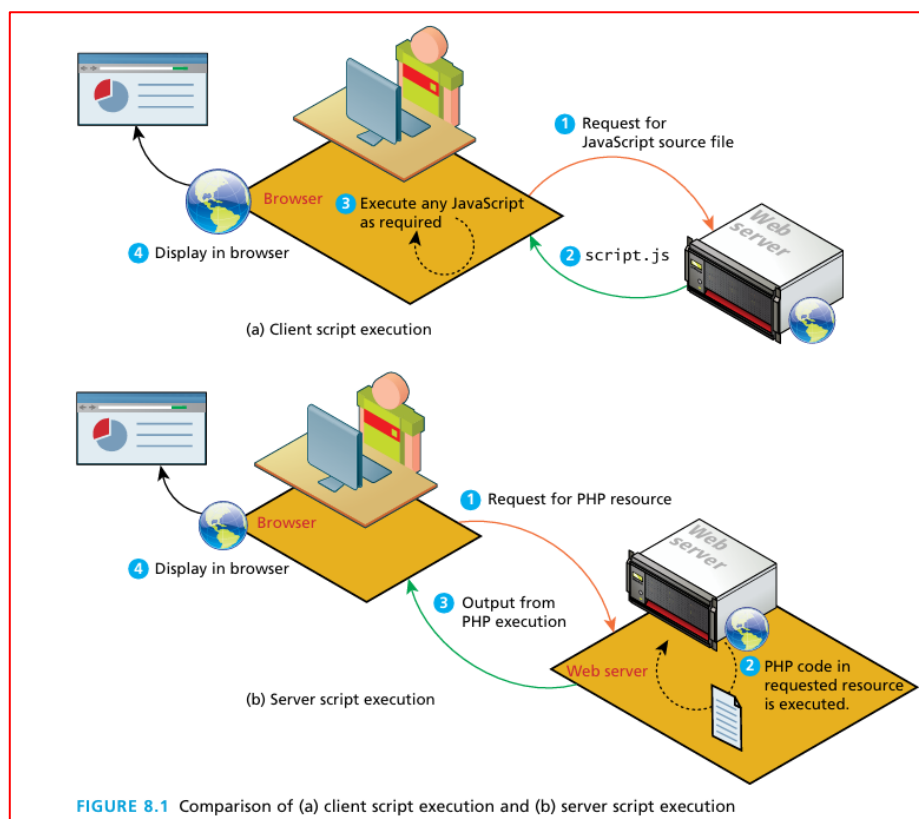
4.1.1 Comparing Client and server scripts

A client-side web programming language (or simply a script). The fundamental difference between client and server scripts is that in a client-side script the code is executed on the client browser, whereas in a server-side script, it is executed on the web server.

Client side JavaScript code is downloaded to the client and is executed there. The server sends the JavaScript (that the user could look at), but you have no guarantee that the script will even execute.

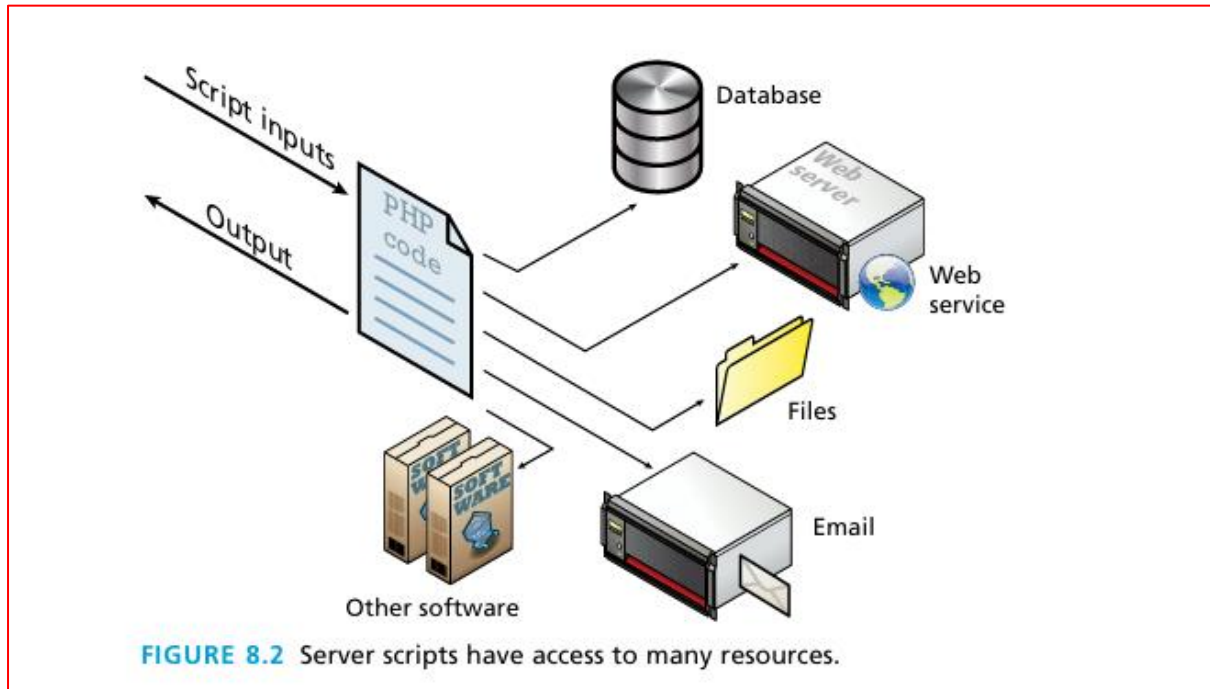
In contrast, server-side source code remains hidden from the client as it is processed on the server. The clients never get to see the code, just the HTML output from the script.

Figure 8.1 illustrates how client and server scripts differ.



4.1.2 Server Side Script Resources

A server-side script can access any resources made available to it by the server. These resources can be categorized as data storage resources, web services, and software applications, as can be seen in Figure 8.2.



The most commonly used resource is data storage, often in the form of a connection to a database management system. A database management system (DBMS) is a software system for storing, retrieving, and organizing large amounts of data.

The term database is often used interchangeably to refer to a DBMS, but it is also used to refer to organized data in general, or even to the files used by the DBMS.

The next suites of resources are web services, often offered by third-party providers. Web services use the HTTP protocol to return XML or other data formats and are often used to extend the functionality of a website.

An example is a geo location service that returns city and country names in response to geographic coordinates.

Finally, there is any additional software that can be installed on a server or accessed via a network connection. Using other software means, server applications can send and receive email, access user authentication services, and use network accessible storage. You could connect a web application to the regular telephone network to send texts or make calls

4.1.3 Comparing Server-Side Technologies

There are several different server-side technologies for creating web applications. The most common include

ASP (Active Server Pages). This was Microsoft's first server-side technology (also called ASP Classic). Like PHP, ASP code (using the VBScript programming language) can be embedded within the HTML; though it supported classes and some object-oriented features, most developers did not make use of these features. ASP programming code is interpreted at run time, hence it can be slow in comparison to other technologies.

ASP.NET. This replaced Microsoft's older ASP technology. ASP.NET is part of Microsoft's .NET Framework and can use any .NET programming language (though C# is the most commonly used). ASP.NET uses an explicitly object-oriented approach that typically takes longer to learn than ASP or PHP, and is often used in larger corporate web application systems. It also uses special markup called web server controls that encapsulate common web functionality such as database-driven lists, form validation, and user registration wizards. A recent extension called ASP.NET MVC makes use of the Model-View Controller design pattern (this pattern will be covered in Chapter 14). ASP.NET pages are compiled into an intermediary file format called MSIL that is analogous to Java's byte-code. ASP.NET then uses a JIT (Just-In-Time) compiler to compile the MSIL into machine executable code so its performance can be excellent. However, ASP.NET is essentially limited to Windows servers.

JSP (Java Server Pages). JSP uses Java as its programming language and like ASP.NET it uses an explicit object-oriented approach and is used in large enterprise web systems and is integrated into the J2EE environment. Since JSP uses the Java Runtime Engine, it also uses a JIT compiler for fast execution time and is cross-platform. While JSP's usage in the web as a whole is small, it has a substantial market share in the intranet environment, as well as with very large and busy sites.

Node.js. This is a more recent server environment that uses JavaScript on the server side, thus allowing developers already familiar with JavaScript to use just a single language for both client-side and server-side development. Unlike the other development technologies listed here, node.js is also its own web server software, thus eliminating the need for Apache, IIS, or some other web server software.

Perl. Until the development and popularization of ASP, PHP, and JSP, Perl was the language typically used for early server-side web development. As a language, it excels in the manipulation of text. It was commonly used in conjunction with the Common Gateway Interface (CGI), an early standard API for communication between applications and web server software.

PHP. Like ASP, PHP is a dynamically typed language that can be embedded directly within the HTML, though it now supports most common object-oriented features, such as classes and inheritance. By default, PHP pages are compiled into an intermediary representation called opcodes that are analogous to Java's byte-code or the .NET Framework's MSIL. Originally, PHP stood for personal home pages, although it now is a recursive acronym that means PHP: Hypertext Processor.

Python. This terse, object-oriented programming language has many uses, including being used to create web applications. It is also used in a variety of web development frameworks such as **Django and Pyramid**.

Ruby on Rails. This is a web development framework that uses the Ruby programming language. Like ASP.NET and JSP, Ruby on Rails emphasizes the use of common software development approaches, in particular the MVC design pattern. It integrates features such as templates and engines that aim to reduce the amount of development work required in the creation of a new site.

All of these technologies share one thing in common: using programming logic, they generate HTML and possibly CSS and JavaScript on the server and send it back to the requesting browser, as shown in Figure 8.3.

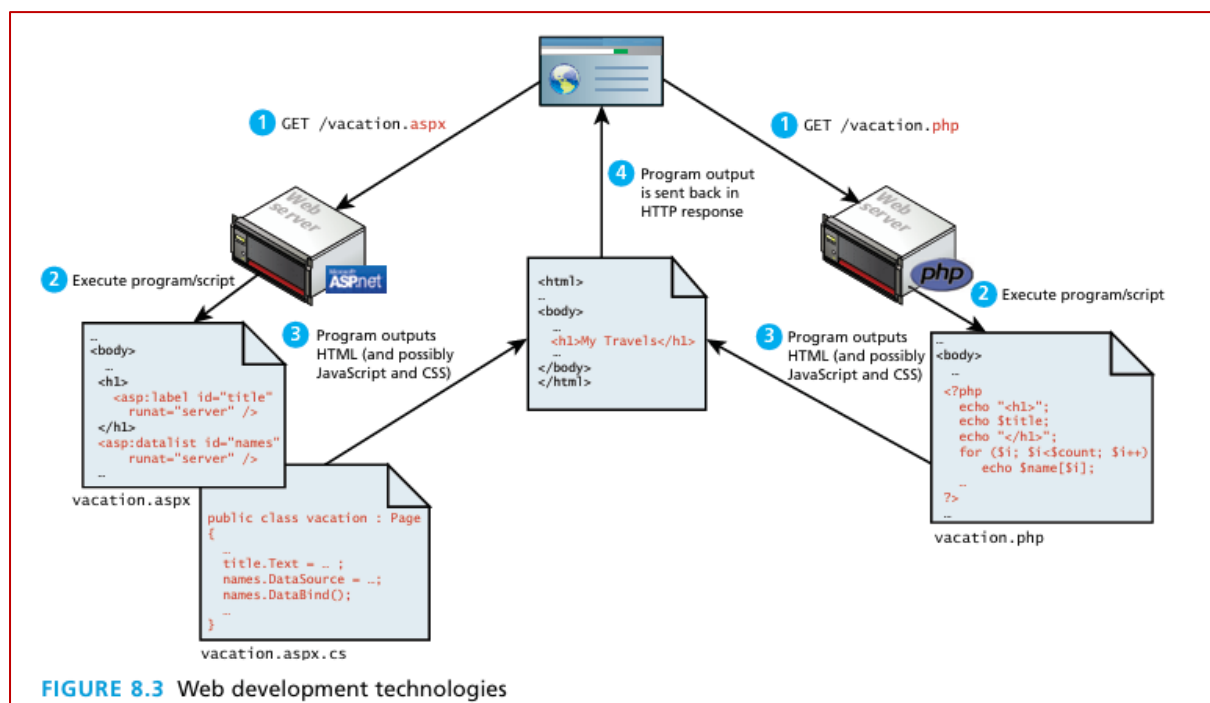


FIGURE 8.3 Web development technologies

Note: Of these server-side technologies, ASP.NET and PHP appear to have the largest market share. ASP.NET tends to be more commonly used for enterprise applications and within intranets. Partly due to the massive user base of WordPress, PHP is the most commonly used web development technology.

4.2 Quick Tour of PHP

Essential features of PHP:

4.2.1 PHP Tags

- The most important fact about PHP is that the programming code can be embedded directly within an HTML file. However, instead of having an .html extension, a PHP file will usually have the extension .php.
- As can be seen in Listing 8.1, PHP programming code must be contained within an opening tag in order to differentiate it from the HTML. The programming code within the tags is interpreted and executed, while any code outside the tags is echoed directly out to the client.

```
<?php
$user = "Randy";
?>
<!DOCTYPE html>
<html>
<body>
<h1>Welcome <?php echo $user; ?></h1>
<p>
The server time is
<?php
echo "<strong>";
echo date("H:i:s");
echo "</strong>";
?>
</p>
</body>
</html>
```

LISTING 8.1 PHP tags

Output:



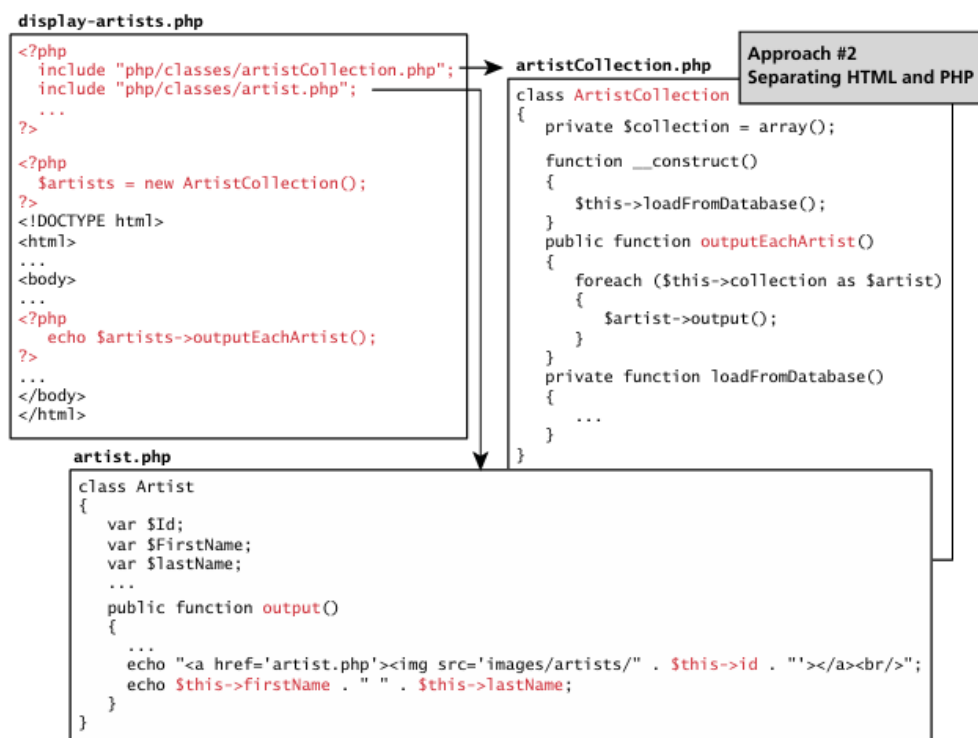
Note:

First method: for a PHP file to have HTML markup and PHP programming woven together. While this is convenient when first learning PHP, as your code becomes more complex, doing so will make your PHP pages very difficult to understand and modify. Indeed, the authors have seen PHP files that are

several thousands of lines long, which are quite a nightmare to maintain. In software design lingo, we would say that such PHP files tightly couple presentation and logic.



Second Method: A layered approach where PHP classes are developed to handle the business and data logic, while smaller presentation PHP files will be used for presenting the information. These presentation scripts will alternate between HTML and PHP, leaving the majority of our PHP files HTML free.



4.2.2 PHP Comments

Programmers are supposed to write documentation to provide other developers (and themselves) guidance on certain parts of a program. In PHP any writing that is a comment is ignored when the script is interpreted, but visible to developers who need to write and maintain the software. The types of comment styles in PHP are:

■ **Single-line comments.** Lines that begin with a # are comment lines and will not be executed.

■ **Multiline (block) comments.** Each PHP script and each function within it are ideal places to include a large comment block. These comments begin with a /* and encompass everything that is encountered until a closing */ tag is found. These tags cannot be nested. A comment block above a function or at the start of a file is a good place to write, in normal language, what this function does. By using the /** tag to open the comment instead of the standard /*, you are identifying blocks of comment that can later be parsed for inclusion in generated documents.

■ **End-of-line comments.** Comments need not always be large blocks of natural language. Sometimes a variable needs a little blurb to tell the developer what it's for, or a complex portion of code needs a few comments to help the programmer understand the logic. Whenever // is encountered in code, everything up to the end of the line is considered a comment. These comments are sometimes preferable to the block comments because they do not interfere with one another, but are unable to span multiple lines of code.

```
<?php

# single-line comment

/*
This is a multiline comment.
They are a good way to document functions or complicated blocks of code
*/

$artist = readDatabase(); // end-of-line comment

?>
```

LISTING 8.3 PHP comments

4.2.3 Variables, Data types and Constants

Variables in PHP are dynamically typed, which means that you as a programmer do not have to declare the data type of a variable. Instead the PHP engine makes a best guess as to the intended type based on what it is being assigned. Variables are also loosely typed in that a variable can be assigned different data types over time.

Creating a variable in PHP: To declare a variable you must preface the variable name with the dollar (\$) symbol. Whenever you use that variable, you must also include the \$ symbol with it. You can assign a value to a variable as in JavaScript’s right-to-left assignment, so creating a variable named count and assigning it the value of 42 would be done with:

```
$count = 42;
```

Note:

1. You should note that in PHP the name of a variable is case-sensitive, so \$count and \$Count are references to two different variables.
2. In PHP, variable names can also contain the underscore character, which is useful for readability reasons.
3. If you do not assign a value to a variable and simply define its name, it will be undefined. You can check to see whether a variable has been set using the isset() function, but what’s important to realize is that there are no “useful” default values in PHP. Since PHP is loosely typed, you should always define your own default values in initialization.

Main data types : Table 8.1 lists the main data types within PHP.

Data Type	Description
Boolean	A logical true or false value
Integer	Whole numbers
Float	Decimal numbers
String	Letters
Array	A collection of data of any type (covered in the next chapter)
Object	Instances of classes

TABLE 8.1 PHP Data Types

Note: however, you do not declare a data type. Instead the PHP engine determines the data type when the variable is assigned a value.

A constant: is somewhat similar to a variable, except a constant’s value never changes . . . in other words it stays constant. A constant can be defined anywhere but is typically defined near the top of a PHP file via the define() function, as shown in Listing 8.4. The define() function generally takes two parameters: the name of the constant and its value. Notice that once it is defined, it can be referenced without using the \$ symbol.

```
<?php

# uppercase for constants is a programming convention
define("DATABASE_LOCAL", "localhost");
define("DATABASE_NAME", "ArtStore");
define("DATABASE_USER", "Fred");
define("DATABASE_PASSWD", "F5^7%ad");
...
# notice that no $ prefaces constant names
$db = new mysqli(DATABASE_LOCAL, DATABASE_NAME, DATABASE_USER,
    DATABASE_NAME);

?>
```

LISTING 8.4 PHP constants

String Escape Sequences:

Sequence	Description
\n	Line feed
\t	Horizontal tab
\\	Backslash
\\$	Dollar sign
\"	Double quote

TABLE 8.2 String Escape Sequences

String literals:

String literals in PHP can be defined using either the single quote or the double quote character. If a literal is defined using double quotes, then you can also specify escape sequences using the backslash. For instance, the string "Good\nMorning" contains a newline character between the two words. Table 8.2 lists some of the common string escape sequences.

4.2.4 Writing Output in PHP:

Remember that PHP pages are programs that output HTML. To output something that will be seen by the browser, you can use the `echo()` function.

```
echo ("hello");
```

There is also an equivalent shortcut version that does not require the parentheses.

```
echo "hello";
```

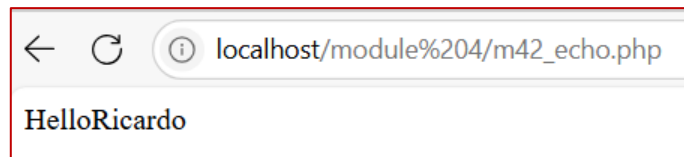
Strings can easily be appended together using the concatenate operator, which is the period (.) symbol. Consider the following code:

```
$username = "Ricardo";  
echo "Hello". $username;
```

example 1: program code

```
<?php  
$username = "Ricardo";  
echo "Hello". $username;  
?>
```

Output:



A screenshot of a web browser window. The address bar shows the URL 'localhost/module%204/m42_echo.php'. Below the address bar, the text 'HelloRicardo' is displayed.

Example 2 : Write a PHP program with the use of quote usage and concatenation approaches.

```
<?php  
$firstName = "Sudarsanan";  
$lastName = "Doss";  
/*  
Example one:  
These two lines are equivalent. Notice that you can reference PHP  
variables within a string literal defined with double quotes.  
The resulting output for both lines is:  
<em>Pablo Picasso</em>  
*/  
echo "<em>" . $firstName . " ". $lastName. "</em>";  
echo "<em> $firstName $lastName </em>";  
/*
```

Example two:

These two lines are also equivalent. Notice that you can use either the single quote symbol or double quote symbol for string literals.

```
*/  
echo "<h1>";  
echo '<h1>';  
/*
```

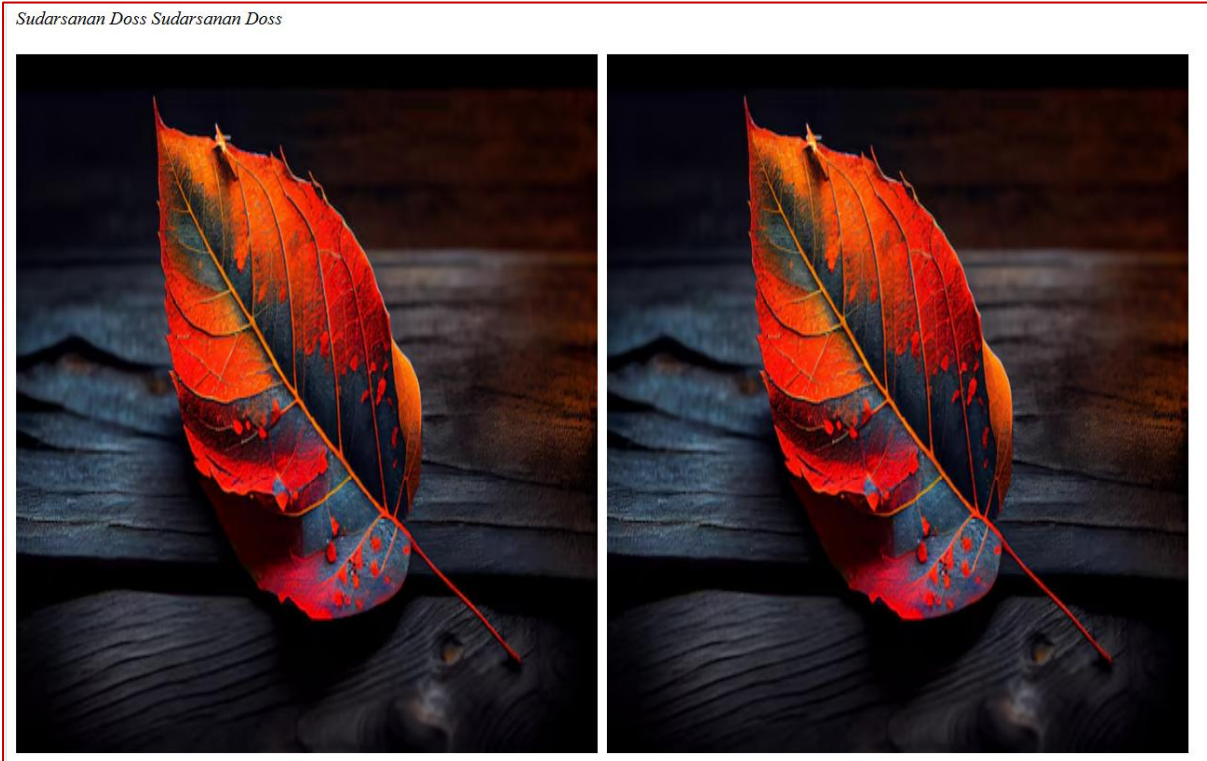
Example three:

These two lines are also equivalent. In the second example, the

escape character (the backslash) is used to embed a double quote within a string literal defined within double quotes.

```
*/  
echo '';  
echo "\t";  
echo "<img src=\"flower.jpg\" width=\"500\" height=\"600\" >"; ?>
```

Output:



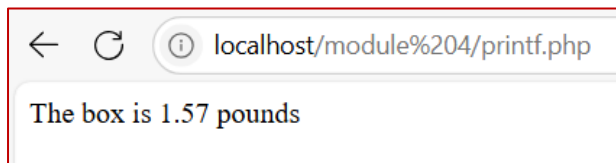
Printf statement:

As the examples in Listing 8.6 illustrate, while echo is quite simple, more complex output can get confusing. As an alternative, you can use the printf() function. This function is derived from the same-named function in the C programming language and includes variations to print to string and files (sprintf, fprintf). The function takes at least one parameter, which is a string, and that string optionally references parameters, which are then integrated into the first string by placeholder substitution. The printf() function also allows a developer to apply special formatting, for instance, specific date/time formats or number of decimal places.

Example:

```
<?php  
$product = "box";  
$weight = 1.56789;  
printf("The %s is %.2f pounds", $product, $weight);  
?>
```

Output:



Each placeholder requires the percent (%) symbol in the first parameter string followed by a type specifier. Common type specifiers are b for binary, d for signed integer, f for float, o for octal, and x for hexadecimal. Precision is achieved in the string with a period (.) followed by a number specifying how many digits should be displayed for floating-point numbers.

4.3 Program Control

Just as with most other programming languages there are a number of conditional and iteration constructs in PHP. There are if and switch, and while, do while, and for loops familiar to most languages as well as the foreach loop.

4.3.1 if ... else Statement

The syntax for conditionals in PHP is almost identical to that of JavaScript. In this syntax the condition to test is contained within () brackets with the body contained in {} blocks.

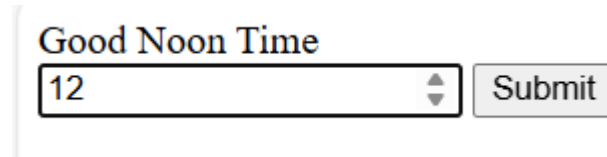
```
<?php
if(isset($_POST['submit']))
{
    $hourOfDay=$_POST['answer'];
    // if statement with condition
    if ( $hourOfDay > 6 && $hourOfDay < 12 ) {
        $greeting = "Good Morning";
        echo "$greeting";
    }
    else if ($hourOfDay == 12) { // optional else if
        $greeting = "Good Noon Time";
        echo "$greeting";
    }
    else {
        // optional else branch
        $greeting = "Good Afternoon or Evening";
        echo "$greeting";
    }
}
?>
<html>
<body>
<form method="post" action="ifelse.php">
```

```

<input type="number" name="answer">
<input type="submit" name="submit" value="Submit">
</form>
</body>
</html>

```

Output:



An alternate form of an if . . . else statement in php is illustrated

This approach will sometimes be used when the body of a conditional contains nothing but markup with no logic, though because it mixes markup and logic.

```

<?php $userStatus="loggedin"; ?>
<?php if ($userStatus == "loggedin") { ?>
    <a href="account.php">Account</a>
    <a href="logout.php">Logout</a>
<?php } else { ?>
    <a href="login.php">Login</a>
    <a href="register.php">Register</a>
<?php } ?>

```

Output:



```

<?php
    // equivalent to the above conditional
    if ($userStatus == "loggedin") {
        echo '<a href="account.php">Account</a> ';
        echo '<a href="logout.php">Logout</a>';
    }
    else {
        echo '<a href="login.php">Login</a> ';
        echo '<a href="register.php">Register</a>';
    }
?>

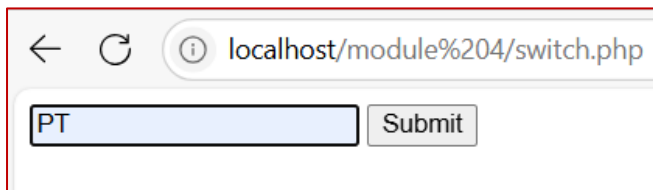
```

4.3.2 Switch ... Case Statement

The switch statement is similar to a series of if . . . else statements. An example using switch is shown in Listing 8.9.

```
<?php
if(isset($_POST['submit']))
{
    $artType=$_POST['answer'];
    switch ($artType) {
        case "PT":
            $output = "Painting";
            echo "$output";
            break;
        case "SC":
            $output = "Sculpture";
            echo "$output";
            break;
        default:
            $output = "Other";
            echo "$output";
    }
}
?>
<html>
<body>
<form method="post" action="switch.php">
<input type="string" name="answer">
<input type="submit" name="submit" value="Submit">
</form>
</body>
</html>
```

Input:



Output



Using series of if ... else Statement

```
// equivalent
if ($artType == "PT")
    $output = "Painting";
else if ($artType == "SC")
    $output = "Sculpture";
else
    $output = "Other";
```

4.3.3 While do ... While

The while loop and the do . . . while loop are quite similar. Both will execute nested statements repeatedly as long as the while expression evaluates to true. In the while loop, the condition is tested at the beginning of the loop; in the do . . . while loop the condition is tested at the end of each iteration of the loop.

```
<?php
echo "example ofwhile loop"."<br>";
$count = 0;
while ($count < 10)
{
    echo $count."\t";
    $count++;
}

echo "<br>". "example of do while loop"."<br>";
$count = 0;
do
{
    echo $count;
    $count++;
} while ($count < 10)
?>
```

```
example ofwhile loop
0 1 2 3 4 5 6 7 8 9
example of do while loop
0123456789
```

4.3.4 for loop in PHP:

The for loop in PHP has the same syntax as the for loop in JavaScript. the for loop contains the same loop initialization, condition, and post-loop operations as in JavaScript.

```
<?php
echo "for loop example". "<br>";
for ($count=0; $count < 10; $count++)
{
    echo $count."<br>";
}
?>
```

Output:

```
for loop example
0
1
2
3
4
5
6
7
8
9
```

Note: There is another type of for loop: the foreach loop. This loop is especially useful for iterating through arrays and so this book will cover foreach loops in the array section.

4.3.5 Alternate syntax for Control structures

PHP has an alternative syntax for most of its control structures (namely, the if, while, for, foreach, and switch statements). In this alternate syntax (shown in Listing 8.12), the colon (:) replaces the opening curly bracket, while the closing

```
<?php if ($userStatus == "loggedin") : ?>
    <a href="account.php">Account</a>
    <a href="logout.php">Logout</a>
<?php else : ?>
    <a href="login.php">Login</a>
    <a href="register.php">Register</a>
<?php endif; ?>
```

LISTING 8.12 Alternate syntax for control structures

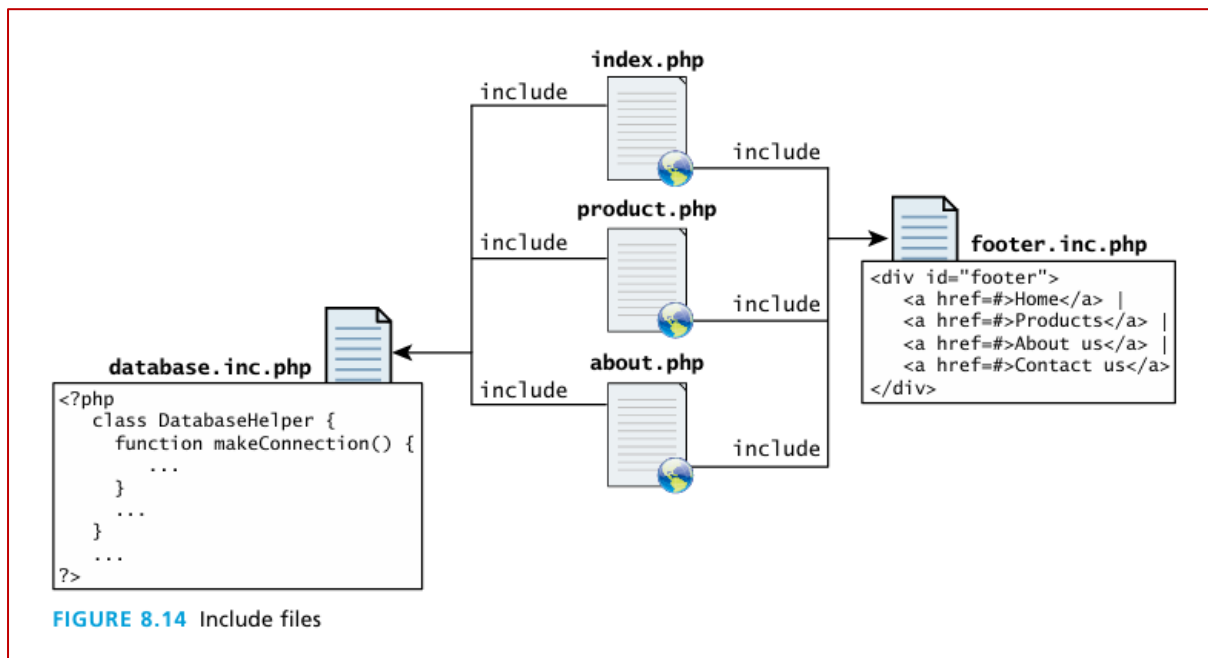
Print some numbers from 1 to 4 using For loops

```
<?php
$i = 1;
for($i=0;$i<=4;$i++) :
    echo $i;
```

```
endfor;  
?>  
Output : 1 2 3 4 .
```

4.3.6 include Files:

PHP does have one important facility that is generally unlike other nonweb programming languages, namely the ability to include or insert content from one file into another.⁵ Almost every PHP page beyond simple practice exercises makes use of this include facility. Include files provide a mechanism for reusing both markup and PHP code, as shown in Figure 8.14.



PHP provides four different statements for including files, as shown below.

```
include "somefile.php";
include_once "somefile.php";
require "somefile.php";
require_once "somefile.php";
```

The difference between `include` and `require` lies in what happens when the specified file cannot be included (generally because it doesn't exist or the server doesn't have permission to access it). With `include`, a warning is displayed and then execution continues. With `require`, an error is displayed and execution stops. The `include_once` and `require_once` statements work just like `include` and `require` but if the requested file has already been included once, then it will not be included again. This might seem an unnecessary addition, but in a complex PHP application written by a team of developers, it can be difficult to keep track of if a given file has been included. It is not uncommon for a PHP page to include a file that includes other files that may include other files, and in such an environment the `include_once` and `require_once` statements are certainly recommended.

scope within include Files Include files appear to provide a type of encapsulation, but it is important to realize that they are the equivalent of copying and pasting, though in this case it is performed by the server. This can be quite clearly seen by considering the scope of code within an include file. Variables defined within an include file will have the scope of the line on which the include occurs. Any variables available at that line in the calling file will be available within the called file. If the include occurs inside a function, then all of the code contained in the called file will behave as though it had been defined inside that function. Thus, for true encapsulation, you will have to use functions (covered next) and classes (covered in the next chapter).

Example:

```
//vars.php
<?php
```

```
$color = 'green';
$fruit = 'apple';
```

```
?>
```

```
//test.php
<?php
include 'vars.php';
echo "A $color $fruit";
?>
```

Output:

```
//vars.php A green apple
```

4.4 Functions

Just as with any language, writing code in the main function (which in PHP is equivalent to coding in the markup between tags) is not a good habit to get into. Having all your code in the main body of a script makes it hard to reuse, maintain, and understand.

As an alternative, PHP allows you to define functions. Just like with JavaScript, a function in PHP contains a small bit of code that accomplishes one thing. These functions can be made to behave differently based on the values of their parameters.

Functions can exist all on their own, and can then be called from anywhere that needs to make use of them, so long as they are in scope.

In PHP there are two types of function: user-defined functions and built-in functions. A user-defined function is one that you the programmer define. A built-in function is one of the functions that come with the PHP environment (or with one of its extensions). One of the real strengths of PHP is its rich library of built-in functions that you can use.

4.4.1 Function syntax

To create a new function you must think of a name for it, and consider what it will do. Functions can return values to the caller, or not return a value. They can be set up to take or not take parameters.

Example:

To illustrate function syntax, let us examine a function called `getNiceTime()`, which will return a formatted string containing the current server time.

```
<?php
/**
 * This function returns a nicely formatted string using the current
 * system time.
 */
function getNiceTime() {
    return date("H:i:s");
}

$output = getNiceTime(); //calling function
echo "the current system time is: ".$output."<br>";

echo " directly call the funtion using echo"."<br>";
echo getNiceTime(); //calling function
?>
```

Output:

```
the current system time is: 05:47:48
directly call the funtion using echo
05:47:48
```

The definition of a function without a return value

```
<?php
/**
 * This function outputs the footer menu
 */
function outputFooterMenu() {
    echo '<div id="footer">';
    echo '<a href=#>Home</a> | <a href=#>Products</a> | ';
    echo '<a href=#>About us</a> | <a href=#>Contact us</a>';
    echo '</div>';
}

outputFooterMenu(); //calling function
?>
```

Output:

[Home](#) | [Products](#) | [About us](#) | [Contact us](#)

4.4.2 using parameters in function

It is more common to define functions with parameters, since functions are more powerful and reusable when their output depends on the input they get. Parameters are the mechanism by which values are passed into functions, and there are some complexities that allow us to have multiple parameters, default values, and to pass objects by reference instead of value.

To define a function with parameters, you must decide how many parameters you want to pass in, and in what order they will be passed. Each parameter must be named.

To illustrate, let us write another version of `getNiceTime()` that takes an integer as a parameter to control whether to show seconds. You will call the parameter `showSeconds`, and write our function as shown in Listing 8.15. Notice that parameters, being a type of variable, must be prefaced with a `$` symbol like any other PHP variable

Example:

```
<?php
/**
 * This function returns a nicely formatted string using the current
 * system time. The showSeconds parameter controls whether or not to
```

```

* include the seconds in the returned string.
*/
function getNiceTime($showSeconds) {
    if ($showSeconds==true)
        return date("H:i:s");
    else
        return date("H:i");
}

echo "this will print seconds". "<br>";
echo getNiceTime(1). "<br>";
echo "will not print seconds". "<br>";
echo getNiceTime(0);
?>

```

Output:

```

this will print seconds
06:08:13
will not print seconds
06:08

```

Note:

1. In fact any nonzero number passed in to the function will be interpreted as true since the parameter is not type specific.
2. In PHP, the signature of a function is based on its name, and not its parameters. Thus it is not possible to do the same function overloading as in other object-oriented languages. PHP does have class method overloading, but it means something quite different than in other object-oriented languages.

parameter Default values

In PHP you can set parameter default values for any parameter in a function. However, once you start having default values, all subsequent parameters must also have defaults.

```

<?php
/**
 * This function returns a nicely formatted string using the current
 * system time. The showSeconds parameter controls whether or not to
 * include the seconds in the returned string.
 */
function getNiceTime($showSeconds=1) {
    if ($showSeconds==true)
        return date("H:i:s");
    else
        return date("H:i");
}

```

```

}

echo "this will print seconds"."<br>";
echo getNiceTime(1)."<br>";
echo "will not print seconds"."<br>";
echo getNiceTime();
?>

```

Output:

```

this will print seconds
06:14:29
will not print seconds
06:14:29

```

passing parameters by Value:

By default, arguments passed to functions are passed by value in PHP. This means that PHP passes a copy of the variable so if the parameter is modified within the function, it does not change the original. Listing 8.17 illustrates a simple example of passing by value

```

<?php
function changeParameter($arg) {
    $arg += 300;
    echo "arg=" . $arg."<br>";
}
$initial = 15;
echo "before paasing value to the function initial value is :". $initial . "<br>";
echo "After paasing value to the function initial value is :";
changeParameter($initial);
echo "After cursor coming to main funtion the initial value is ".$initial."<br>";

?>

```

Output:

```

before paasing value to the function initial value is :15
After paasing value to the function initial value is :arg=315
After cursor coming to main funtion the initial value is 15

```

Notice that even though the function modifies the parameter value, the contents of the variable passed to the function remain unchanged after the function has been called.

passing parameters by reference:

passed by reference, which will allow a function to change the contents of a passed variable. A parameter passed by reference points the local variable to the same place as the original, so if the function changes it, the original variable is changed as well. The mechanism in PHP to specify that a parameter is passed by reference is to add an ampersand (&) symbol next to the parameter name in the function declaration. Listing 8.18 illustrates an example of passing by reference.

Example:

```
<?php
function changeParameter(&$arg) {
    $arg += 300;
    echo "arg=" . $arg."<br>";
}
$initial = 15;
echo "before paasing value to the function initial value is :" . $initial . "<br>";
echo "After paasing value to the function initial value is :";
changeParameter($initial);
echo "After cursor coming to main funtion the initial value is ".$initial."<br>";

?>
```

Output:

```
before paasing value to the function initial value is :15
After paasing value to the function initial value is :arg=315
After cursor coming to main funtion the initial value is 315
```

4.4.3 variable scope within Functions

It will come as no surprise that all variables defined within a function (such as parameter variables) have function scope, meaning that they are only accessible within the function. It might be surprising though to learn that any variables created outside of the function in the main script are unavailable within a function. For instance, in the following example, the output of the echo within the function is 0 and not 56 since the reference to \$count within the function is assumed to be a new variable named \$count with function scope.

```
<?php
$count= 56;
function testScope() {
    echo $count;
    // outputs 0 or generates run-time warning/error
}
testScope();
```

```
echo $count;
// outputs 56
?>
```

Output:

Warning: Undefined variable \$count in C:\xampp\htdocs\module 4\scopevar.php on line 4
56

While variables defined in the main script are said to have global scope, unlike in other programming languages, a global variable is not, by default, available within functions. Of course, in the above example, one could simply have passed \$count to the function.

```
$count= 56;

function testScope() {
    global $count;
    echo $count;    // outputs 56
}

testScope();
echo $count;    // outputs 56
```

LISTING 8.19 Using the global keyword

Note:

1. **superglobal variables** in PHP, which are used for accessing query string data, server data, and session storage. One of these is the \$GLOBALS associative array, which is always available and is a convenient storage location for any data that must be available globally.
2. It covers PHP arrays, from the most basic all the way through to superglobal arrays, which are essential for almost any PHP web application. The chapter ends with a look at file processing in PHP, where you will learn to handle file uploads, as well as read and write text files directly

4.5 Arrays

Like most other programming languages, PHP supports arrays. In general, an array is a data structure that allows the programmer to collect a number of related elements together in a single variable. Unlike most other programming languages, in PHP an array is actually an ordered map, which associates each value in the array with a key.

Figure 9.1 illustrates a PHP array with five strings containing day abbreviations.

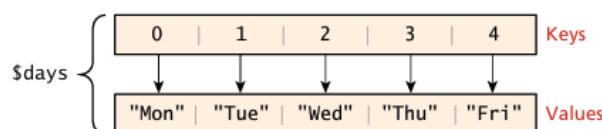


FIGURE 9.1 Visualization of a key-value array

Array keys in most programming languages are limited to integers, start at 0, and go up by 1. In PHP, keys must be either integers or strings and need not be sequential.

Array values, unlike keys, are not restricted to integers and strings. They can be any object, type, or primitive supported in PHP. You can even have objects of your own types, so long as the keys in the array are integers and strings.

4.5.1 Defining and accessing an array

- Let us begin by considering the simplest array, which associates each value inside of it with an integer index (starting at 0). The following declares an empty array named days:

```
$days = array();
```

- To define the contents of an array as strings for the days of the week as shown in Figure 9.1

you declare it with a comma-delimited list of values inside the () braces using either of two following syntaxes

```
$days = array("Mon","Tue","Wed","Thu","Fri");  
$days = ["Mon","Tue","Wed","Thu","Fri"];    // alternate syntax
```

Note: In these examples, because no keys are explicitly defined for the array, the default key values are 0, 1, 2, . . . , n. Notice that you do not have to provide a size for the array: arrays are dynamically sized as elements are added to them.

- Elements within a PHP array are accessed in a manner similar to other programming languages, that is, using the familiar square bracket notation. The code example below echoes the value of our \$days array for the key=1, which results in output of Tue.

```
echo "Value at index 1 is ". $days[1]; // index starts at zero
```

You could also define the array elements individually using this same square bracket notation:

```
$days = array();  
$days[0] = "Mon";  
$days[1] = "Tue";  
$days[2] = "Wed";  
  
// also alternate approach  
$daysB = array();  
$daysB[] = "Mon";  
$daysB[] = "Tue";  
$daysB[] = "Wed";
```

In PHP, you are also able to explicitly define the keys in addition to the values. This allows you to use keys other than the classic 0, 1, 2, . . . , n to define the indexes of an array. For clarity, the exact same array defined above and shown in Figure 9.1 can also be defined more explicitly by specifying the keys and values as shown in Figure 9.2

```
$days = array(0 => "Mon", 1 => "Tue", 2 => "Wed", 3 => "Thu", 4 => "Fri");
```

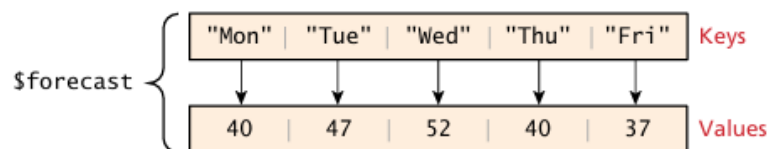
key
└─┘
value

FIGURE 9.2 Explicitly assigning keys to array elements

Array with strings as keys and integers as values

```
$forecast = array("Mon" => 40, "Tue" => 47, "Wed" => 52, "Thu" => 40, "Fri" => 37);
```

key
└─┘
value



```
echo $forecast["Tue"]; // outputs 47
echo $forecast["Thu"]; // outputs 40
```

FIGURE 9.3 Array with strings as keys and integers as values

Example:

```
<?php
$season=array("summer","winter","spring","autumn");
echo "Season are: $season[0], $season[1], $season[2] and $season[3]";
?>
```

Output:

Season are: summer, winter, spring and autumn

PHP Associative Array

```
<?php
$salary=array("Sonoo"=>"350000","John"=>"450000","Kartik"=>"200000");
echo "Sonoo salary: ".$salary["Sonoo"]."<br/>";
echo "John salary: ".$salary["John"]."<br/>";
echo "Kartik salary: ".$salary["Kartik"]."<br/>";
?>
```

Output:

```
Sonoo salary: 350000  
John salary: 450000  
Kartik salary: 200000
```

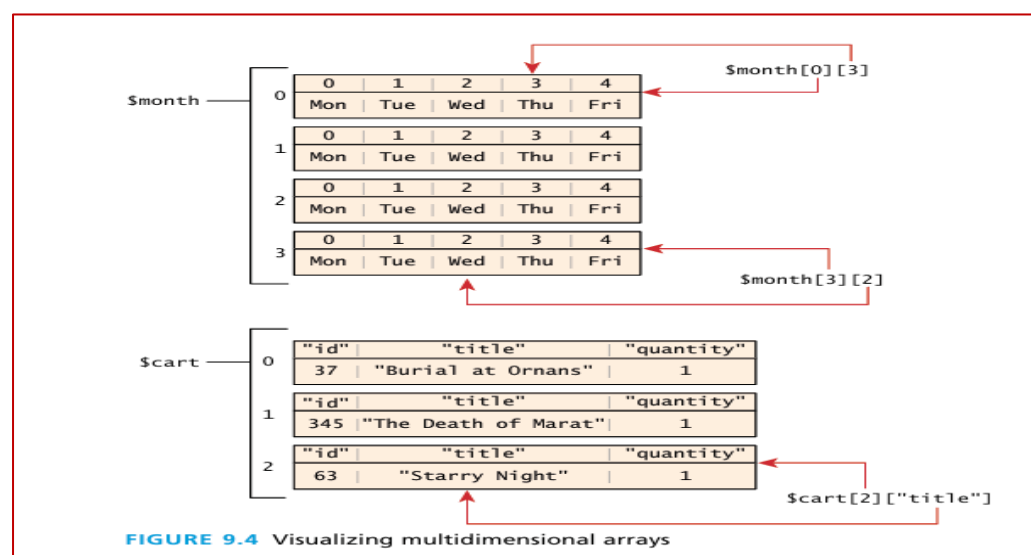
Example 3:

```
<?php  
$salary["Sonoo"]="350000";  
$salary["John"]="450000";  
$salary["Kartik"]="200000";  
echo "Sonoo salary: ".$salary["Sonoo"]."<br/>";  
echo "John salary: ".$salary["John"]."<br/>";  
echo "Kartik salary: ".$salary["Kartik"]."<br/>";  
?>
```

```
Sonoo salary: 350000  
John salary: 450000  
Kartik salary: 200000
```

4.5.2 Multidimensional arrays

PHP also supports multidimensional arrays. Recall that the values for an array can be any PHP object, which includes other arrays. Listing 9.1 illustrates the creation of two different multidimensional arrays (each one contains two dimensions).



Example:

```
<?php
$month = array
(
    array("Mon","Tue","Wed","Thu","Fri"),
    array("Mon","Tue","Wed","Thu","Fri"),
    array("Mon","Tue","Wed","Thu","Fri"),
    array("Mon","Tue","Wed","Thu","Fri")
);
echo $month[0][3];
?>
```

Output:

Thu

```
<?php
$month = array
(
    array("Mon","Tue","Wed","Thu","Fri"),
    array("Mon","Tue","Wed","Thu","Fri"),
    array("Mon","Tue","Wed","Thu","Fri"),
    array("Mon","Tue","Wed","Thu","Fri")
);
echo $month[1][4];
?>
```

Output:

Fri

4.5.3 iterating through an array

One of the most common programming tasks that you will perform with an array is to iterate through its contents. Listing 9.2 illustrates how to iterate and output the content of the \$days array using the built-in function count() along with examples using while, do while, and for loops.

Example:

```
<?php
$days = array("Mon","Tue","Wed","Thu","Fri");
echo "using while loop " . "<br>";
$i=0;
while ($i < count($days)) {
    echo $days[$i] . "<br>";
    $i++;
}
echo "using do while loop " . "<br>";
```

```

$i=0;
do {
    echo $days[$i] . "<br>";
    $i++;
} while ($i < count($days));
echo "using for loop " . "<br>";
for ($i=0; $i<count($days); $i++) {
    echo $days[$i] . "<br>";
}
?>

```

Output:

```

using while loop
Mon
Tue
Wed
Thu
Fri
using do while loop
Mon
Tue
Wed
Thu
Fri
using for loop
Mon
Tue
Wed
Thu
Fri

```

4.5.4 Iterating through an associative array using a foreach loop

The challenge of using the classic loop structures is that when you have non sequential integer keys (i.e., an associative array), you can't write a simple loop that uses the `$i++` construct. To address the dynamic nature of such arrays, you have to use iterators to move through such an array. This iterator concept has been woven into the `foreach` loop and illustrated for the `$forecast` array in

Example:

```

<?php
$forecast = array("Mon" => 40, "Tue" => 47, "Wed" => 52, "Thu" => 40, "Fri" => 37);
echo "foreach: iterating through the values" . "<br>";
foreach ($forecast as $value) {
    echo $value . "<br>";
}
echo "foreach: iterating through the values AND the keys" . "<br>";
foreach ($forecast as $key => $value) {
    echo "day : " . $key . " = " . $value . "\t";
}
?>

```

Output:

```
foreach: iterating through the values
40
47
52
40
37
foreach: iterating through the values AND the keys
day :Mon=40 day :Tue=47 day :Wed=52 day :Thu=40 day :Fri=37
```

4.5.5 Adding and Deleting elements

Adding array element:

In PHP, arrays are dynamic, that is, they can grow or shrink in size. An element can be added to an array simply by using a key/index that hasn't been used, as shown below:

```
$days[5] = "Sat";
```

Example:

```
<?php
$days = array("Mon","Tue","Wed","Thu","Fri");
echo "Dynamically add the array element".<br>;
$days[5] = "Sat";
print_r($days);
?>
```

Output:

```
Dynamically add the array element
Array ( [0] => Mon [1] => Tue [2] => Wed [3] => Thu [4] => Fri [5] => Sat )
```

Note:

1. What will be the output of the **print_r()**? It will show that our array now contains the following: Array ([0] => Mon [1] => Tue [2] => Wed [3] => Thu [4] => Fri [5] => Sat).
2. If we try referencing \$days[6], for instance, it will return a NULL value, which is a special PHP value that represents a variable with no value.

Deleting array elements:

You can also create “gaps” by explicitly deleting array elements using the `unset()` function, as shown in Listing 9.4.

```
<?php
$days = array("Mon","Tue","Wed","Thu","Fri");
unset($days[2]);
unset($days[3]);
echo "After deleted the array element of pos 2 and pos 3".<br>;
print_r($days); // outputs: Array ( [0] => Mon [1] => Tue [4] => Fri )
echo "<br>". "alternate method to display After deleted the array element of pos 2 and pos 3".<br>;
$days = array_values($days);
print_r($days); // outputs: Array ( [0] => Mon [1] => Tue [2] => Fri )
?>
```

Output:

```
After deleted the array element of pos 2 and pos 3
Array ( [0] => Mon [1] => Tue [4] => Fri )
alternate method to display After deleted the array element of pos 2 and pos 3
Array ( [0] => Mon [1] => Tue [2] => Fri )
```

Checking if a value exists:

Since array keys need not be sequential, and need not be integers, you may run into a scenario where you want to check if a value has been set for a particular key. As with undefined null variables, values for keys that do not exist are also undefined. To check if a value exists for a key, you can therefore use the `isset()` function, which returns true if a value has been set, and false otherwise. Listing 9.5 defines an array with noninteger indexes, and shows the result of asking `isset()` on several indexes.

Example:

```
<?php
$oddKeys = array (1 => "hello", 3 => "world", 5 => "!");
if (isset($oddKeys[0])) {
    // The code below will never be reached since $oddKeys[0] is not set!
```

```

    echo "there is something set for key 0". $oddKeys[0];
}
if (isset($oddKeys[1])) {
}
// This code will run since a key/value pair was defined for key 1
echo "there is something set for key 1, namely ". $oddKeys[1]
?>

```

Output:

there is something set for key 1, namely hello

4.5.6 Array sorting

One of the major advantages of using a mature language like PHP is its built-in functions. There are many built-in sort functions, which sort by key or by value. To sort the \$days array by its values you would simply use:

```
sort($days);
```

As the values are all strings, the resulting array would be:

```
Array ([0] => Fri [1] => Mon [2] => Sat [3] => Sun [4] => Thu
      [5] => Tue [6] => Wed)
```

However, such a sort loses the association between the values and the keys!

A better sort, one that would have kept keys and values associated together, is:

```
asort($days);
```

The resulting array in this case is:

```
Array ([4] => Fri [0] => Mon [5] => Sat [6] => Sun [3] => Thu
      [1] => Tue [2] => Wed)
```

4.5.7 More array Operations

In addition to the powerful sort functions, there are other convenient functions you can use on arrays. It does not make sense to reinvent the wheel when valid, efficient functions have already been written for you. While we will not go into detail about each one, here is a brief description of some key array functions:

- **array_keys(\$someArray):** This method returns an indexed array with the values being the *keys* of \$someArray.

For example, `print_r(array_keys($days))` outputs

```
Array ( [0] => 0 [1] => 1 [2] => 2 [3] => 3 [4] => 4 )
```

- **array_values(\$someArray):** Complementing the above `array_keys()` function, this function returns an indexed array with the values being the *values* of \$someArray.

For example, `print_r(array_values($days))` outputs

```
Array ( [0] => Mon [1] => Tue [2] => Wed [3] => Thu [4] => Fri )
```

- **array_rand(\$someArray, \$num=1):** Often in games or widgets you want to select a random element in an array. This function returns as many random keys as are requested. If you only want one, the key itself is returned; otherwise, an array of keys is returned.

For example, `print_r(array_rand($days,2))` might output:

```
Array (3, 0)
```

- **array_reverse(\$someArray):** This method returns \$someArray in reverse order. The passed \$someArray is left untouched.

For example, `print_r(array_reverse($days))` outputs:

```
Array ( [0] => Fri [1] => Thu [2] => Wed [3] => Tue [4] => Mon )
```

- **array_walk(\$someArray, \$callback, \$optionalParam):** This method is extremely powerful. It allows you to call a method (\$callback), for each value in \$someArray. The \$callback function typically takes two parameters, the value first, and the key second. An example that simply prints the value of each element in the array is shown below.

```
$someA = array("hello", "world");  
array_walk($someA, "doPrint");  
function doPrint($value,$key){  
    echo $key . ": " . $value;  
}
```

- **in_array(\$needle, \$haystack):** This method lets you search array \$haystack for a value (\$needle). It returns true if it is found, and false otherwise.
- **shuffle(\$someArray):** This method shuffles \$someArray. Any existing keys are removed and \$someArray is now an indexed array if it wasn't already.

4.5.8 Superglobal Arrays

PHP uses special predefined associative arrays called superglobal variables that allow the programmer to easily access HTTP headers, query string parameters, and other commonly needed information (see Table 9.1). They are called superglobal because these arrays are always in scope and always exist, ready for the programmer to access or modify them without having to use the global keyword

Name	Description
\$GLOBALS	Array for storing data that needs superglobal scope
\$_COOKIES	Array of cookie data passed to page via HTTP request
\$_ENV	Array of server environment data
\$_FILES	Array of file items uploaded to the server
\$_GET	Array of query string data passed to the server via the URL
\$_POST	Array of query string data passed to the server via the HTTP header

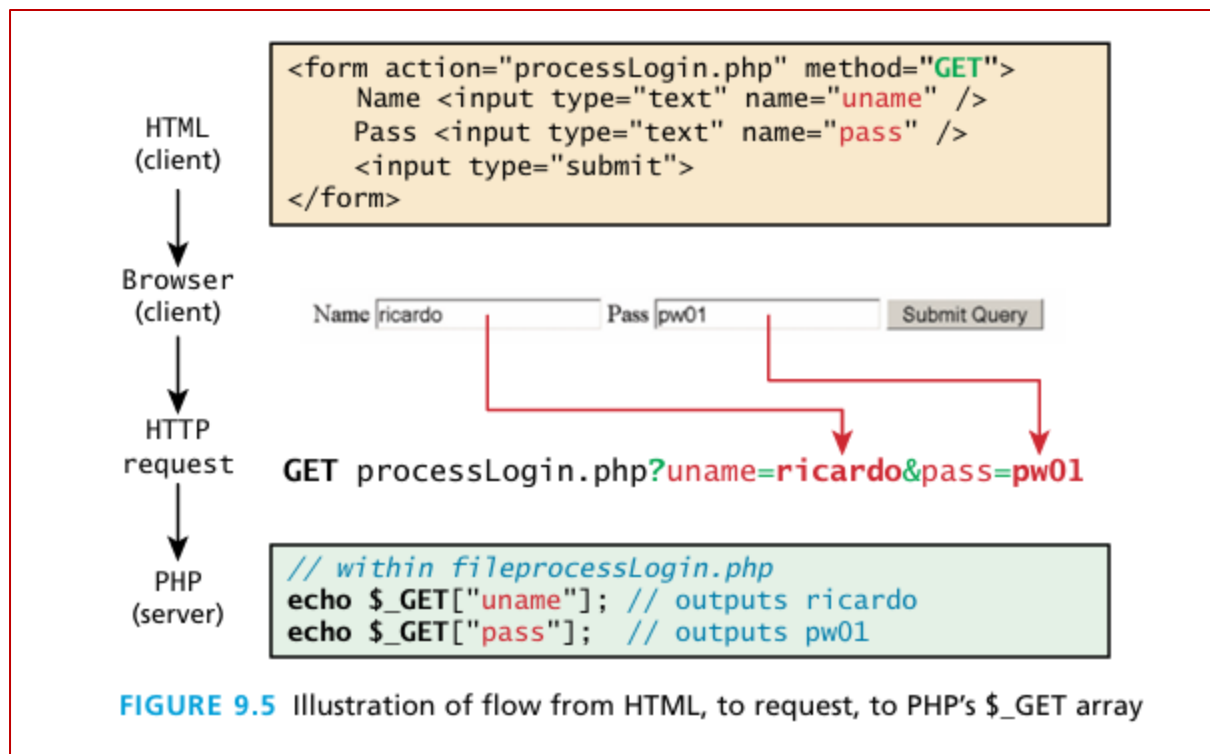
Name	Description
\$_REQUEST	Array containing the contents of \$_GET, \$_POST, and \$_COOKIES
\$_SESSION	Array that contains session data
\$_SERVER	Array containing information about the request and the server

TABLE 9.1 Superglobal Variables

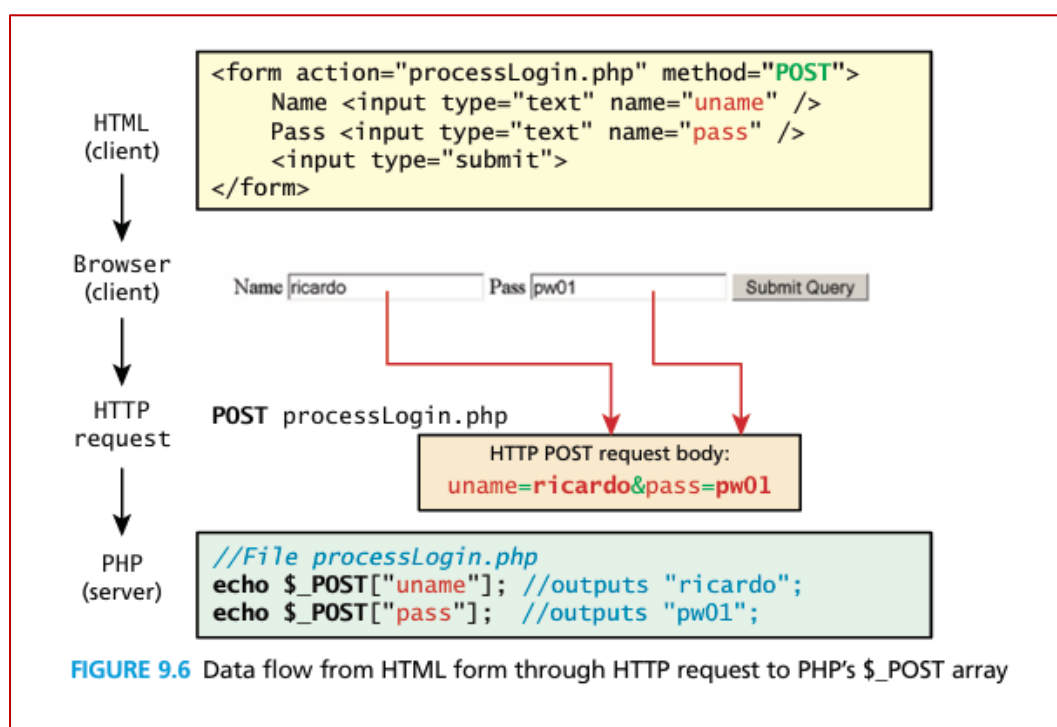
4.5.8.1 \$_GET and \$_POST Superglobal Arrays

The \$_GET and \$_POST arrays are the most important superglobal variables in PHP since they allow the programmer to access data sent by the client in a query string.

That data is formatted such that each value is associated with a name defined in the form. If the form was submitted using an HTTP GET request, then the resulting URL will contain the data in the query string. PHP will populate the superglobal \$_GET array using the contents of this query string in the URL. Figure 9.5 illustrates the relationship between an HTML form, the GET request, and the values in the \$_GET array.



If the form was sent using HTTP POST, then the values would not be visible in the URL, but will be sent through HTTP POST request body. From the PHP programmer's perspective, almost nothing changes from a GET data post except that those values and keys are now stored in the `$_POST` array. This mechanism greatly simplifies accessing the data posted by the user, since you need not parse the query string or the POST request headers. Figure 9.6 illustrates how data from a HTML form using POST populates the `$_POST` array in PHP.



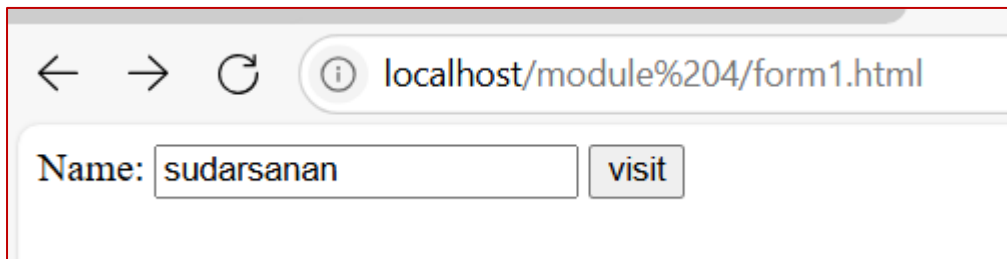
Example 1:

Client side getting the data using html language with help of GET method

File Name: form1.html

```
<html>
<body>
<form action="welcome.php" method="get">
  Name: <input type="text" name="name"/>
        <input type="submit" value="visit"/>
</form>
</body>
</html>
```

Output:



← → ↻ ⓘ localhost/module%204/form1.html

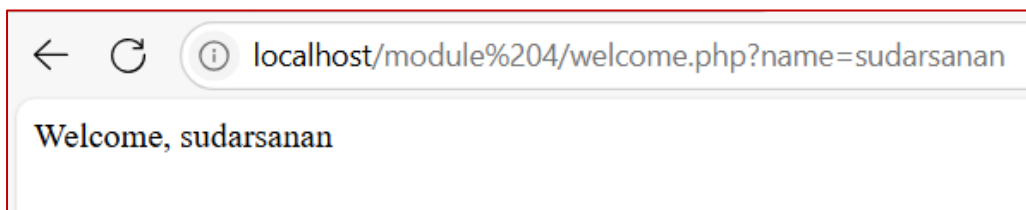
Name:

Service side receive the data using php language with help of superglobal array variable \$_GET().

FILE NAME: welcome.php

```
<?php
$name=$_GET["name");//receiving name field value in $name variable
echo "Welcome, $name";
?>
```

Output:



← ↻ ⓘ localhost/module%204/welcome.php?name=sudarsanan

Welcome, sudarsanan

Example 2:

Client side getting the data using html language with help of POST method

File name: form2.html

```
<form action="login.php" method="post">

<table>

<tr><td>Name:</td><td> <input type="text" name="name"/></td></tr>

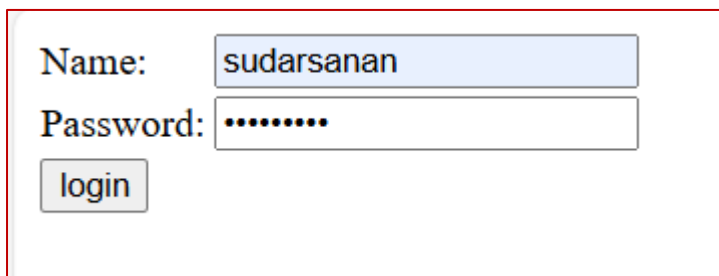
<tr><td>Password:</td><td> <input type="password" name="password"/></td></tr>

<tr><td colspan="2"><input type="submit" value="login"/> </td></tr>

</table>

</form>
```

Output:



Name: sudarsanan

Password:

login

Service side receive the data using php language with help of superglobal array variable \$_POST().

```
<?php

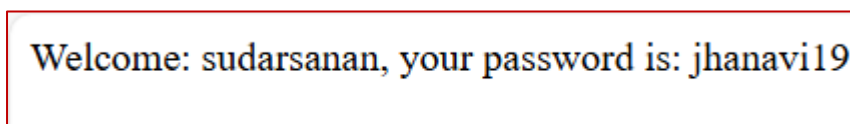
$name=$_POST["name");//receiving name field value in $name variable

$password=$_POST["password");//receiving password field value in $password variable

echo "Welcome: $name, your password is: $password";

?>
```

Output:



Welcome: sudarsanan, your password is: jhanavi19

Working with Databases

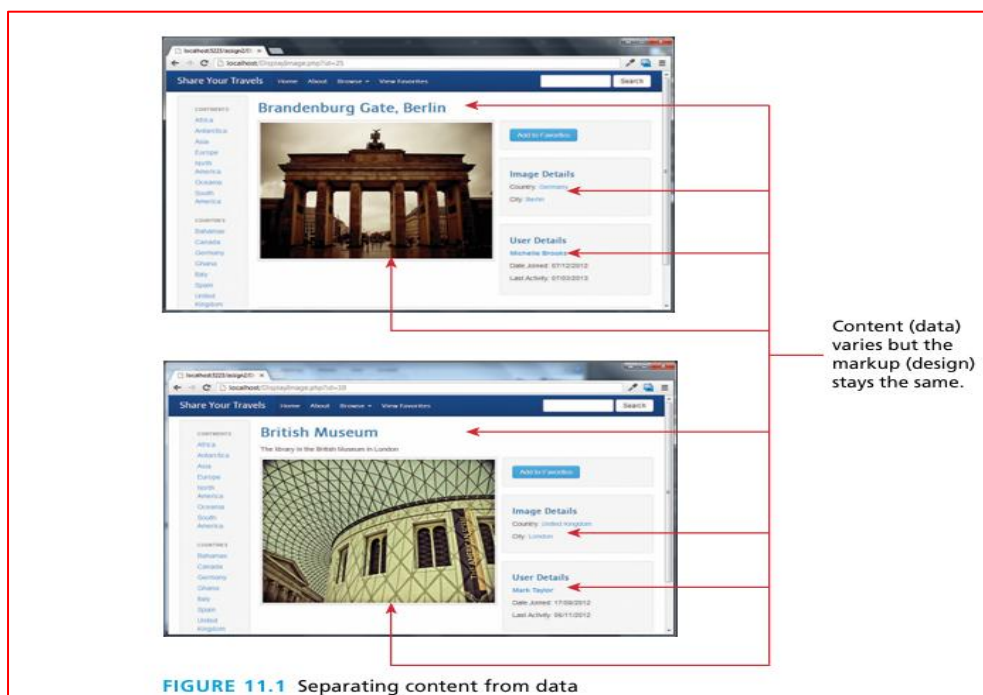
5.1 Databases and Web Development

Almost every dynamic website makes use of some type of server-based data source. The web application development to store data in database using MySQL source code is openly available, it is now owned by Oracle Corporation. There are many other open source and proprietary relational DBMS alternates to MySQL, such as PostgreSQL,² Oracle Database,³ IBM DB2,⁴ and Microsoft SQL Server. All of these database management systems are capable of managing large amounts of data, maintaining data integrity, responding to many queries, creating indexes and triggers, and more.

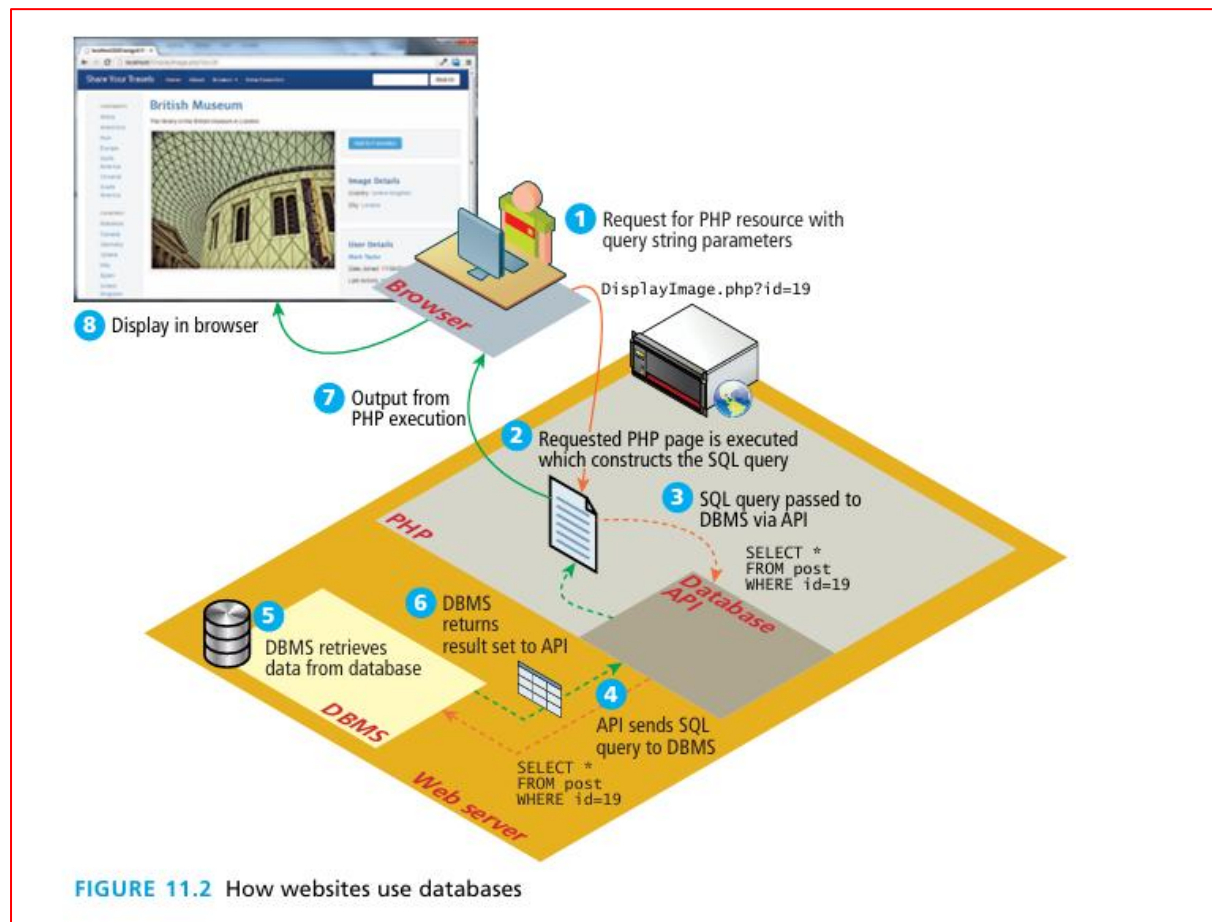
the term database to refer to both the software (i.e., to the DBMS) and to the data that is managed by the DBMS.

5.1.1 the role of Databases in Web Development

The reason that databases are such an essential feature of real-world websites is that they provide a way to implement one of the most important software design principles: namely, that one should separate that which varies from that which stays the same. In the context of the web, sites typically display different content on different pages but those different pages share similar user interface elements, or even have an identical visual design, as shown in Figure 11.1.



In such a case the visual appearance (i.e., the HTML and CSS) is that which stays the same, while the data content is that which varies. So by placing the content into a database, you can programmatically “insert” the content into the markup. The program (in our case written in PHP) determines which data to display, often from information in the GET or POST query string, and then uses a database API to interact with the database, as shown in Figure 11.2.



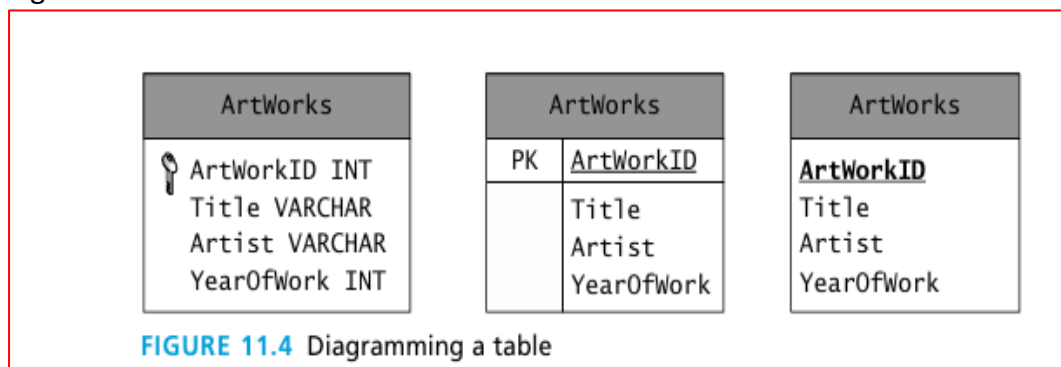
5.1.2 Database Design

- In a relational database, a database is composed of one or more tables. A table is the principal unit of storage in a database. Each table in a database is generally modeled after some type of real-world entity, such as a customer or a product (though as we will see, some tables do not correspond to real-world entities but are used to relate entities together).
- A table is a two-dimensional container for data that consists of records (rows); each record has the same number of columns, which are more specifically called fields, which contain the actual data.
- Each table will have one (or sometimes more than one) special field called a primary key that is used to uniquely identify each record in a table. Figure 11.3 illustrates these different terms.

Field names	ArtWorkID	Title	Artist	YearOfWork
Records	345	The Death of Marat	David	1793
	400	The School of Athens	Raphael	1510
	408	Bacchus and Ariadne	Titian	1520
	425	Girl with a Pearl Earring	Vermeer	1665
	438	Starry Night	Van Gogh	1889

FIGURE 11.3 A database table

- Figure 11.4 illustrates several different ways to visually represent the table shown in Figure 11.3.



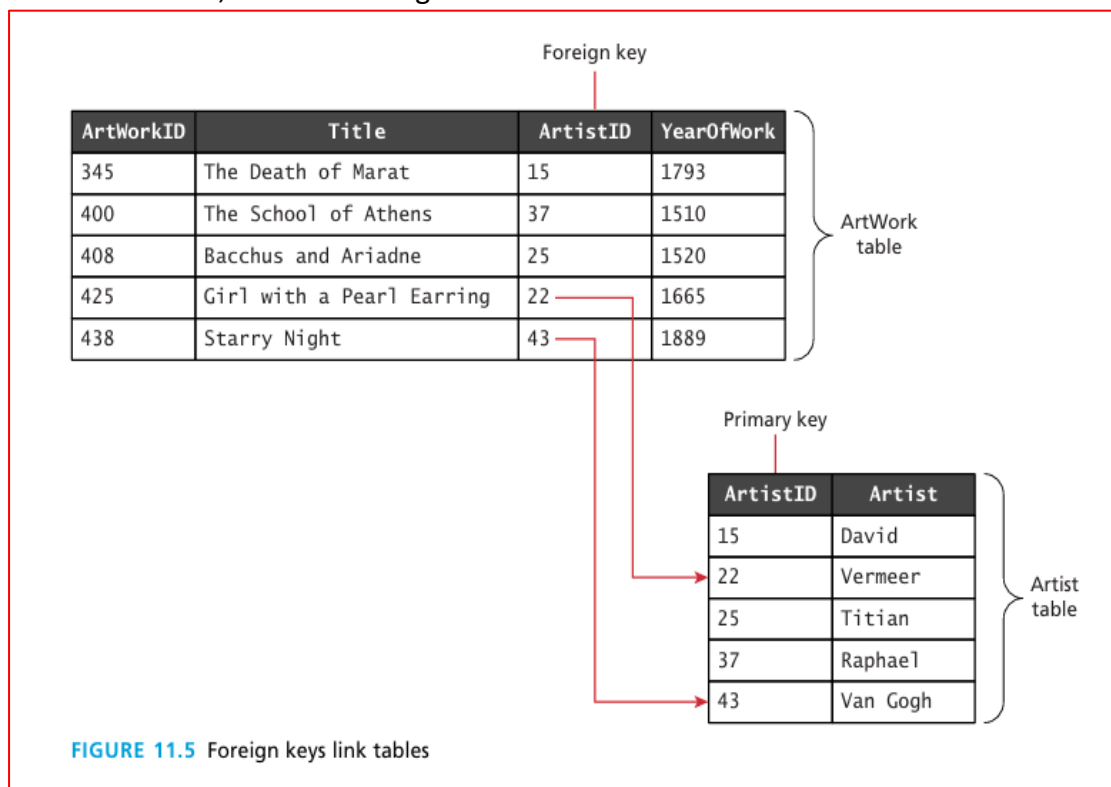
Notice that the table name appears at the top of the table box in all three examples. They differ in how they represent the primary key. The first example also includes the data type of the field, which will be covered shortly.

- One of the strengths of a database in comparison to more open and flexible file formats such as spreadsheets or text files is that a database can enforce rules about what can be stored. This provides data integrity and potentially can reduce the amount of data duplication, which are two of the most important advantages of using databases.
- This is partly achieved through the use of data types that are akin to those in a programming language, some of which are listed in Table 11.1.

Type	Description
BIT	Represents a single bit for Boolean values. Also called <code>BOOLEAN</code> or <code>BOOL</code> .
BLOB	Represents a binary large object (which could, for example, be used to store an image).
CHAR(n)	A fixed number of characters (n = the number of characters) that are padded with spaces to fill the field.
DATE	Represents a date. There are also <code>TIME</code> and <code>DATETIME</code> data types.
FLOAT	Represents a decimal number. There are also <code>DOUBLE</code> and <code>DECIMAL</code> data types.
INT	Represents a whole number. There is also a <code>SMALLINT</code> data type.
VARCHAR(n)	A variable number of characters (n = the maximum number of characters) with no space padding.

TABLE 11.1 Common Database Table Data Types

- One of the most important ways that data integrity is achieved in a database is by separating information about different things into different tables. Two tables can be related together via foreign keys, which is a field that is the same as the primary key of another table, as shown in Figure 11.5.



Tables that are linked via foreign keys are said to be in a relationship. Most often, two related tables will be in a one-to-many relationship. In this relationship, a single record in Table A (e.g., an art work table) can have one or more matching records in Table B (e.g., artist table), but a record in Table B has only one matching record in Table A.

- This is the most common and important type of relationship. Figure 11.6 illustrates some of the different ways of visually representing a one-to-many relationship.

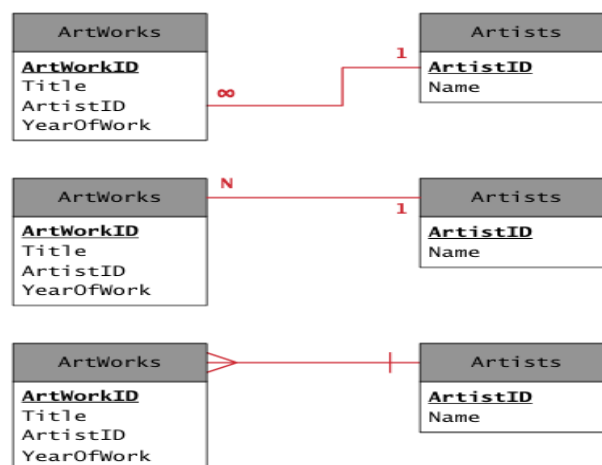
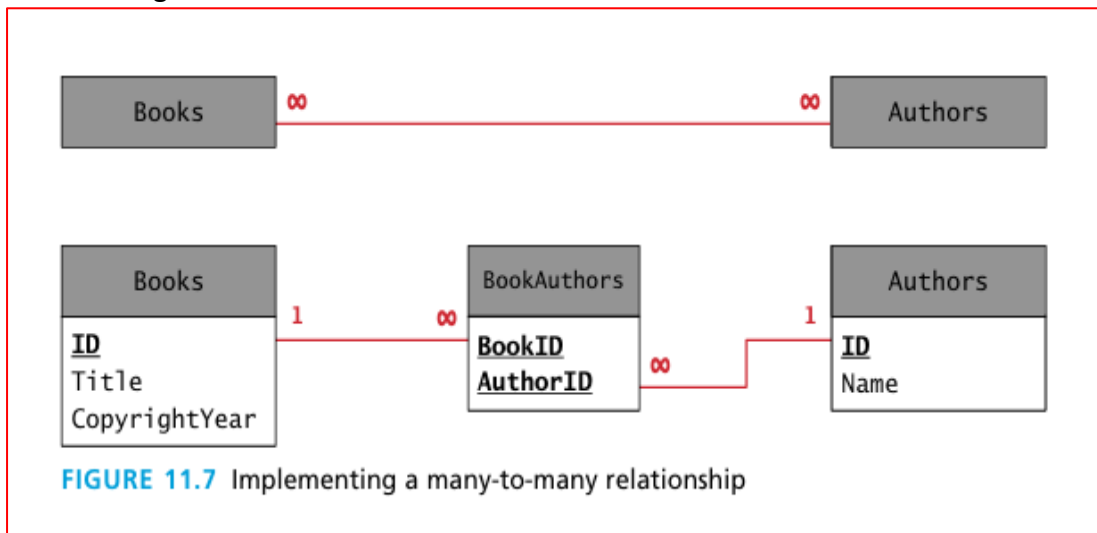


FIGURE 11.6 Diagramming a one-to-many relationship

- There are two other table relationships: the one-to-one relationship and the many-to-many relationship. Since the information in a one-to-one relationship could be stored in a single table, they are encountered less often and are typically used for performance or security reasons. Many-to-many relationships are, on the other hand, quite common. For instance, a single book may be written by multiple authors; a single author may write multiple books. Many-to-many relationships are usually implemented by using an intermediate table with two one-to-many relationships, as shown in Figure 11.7.

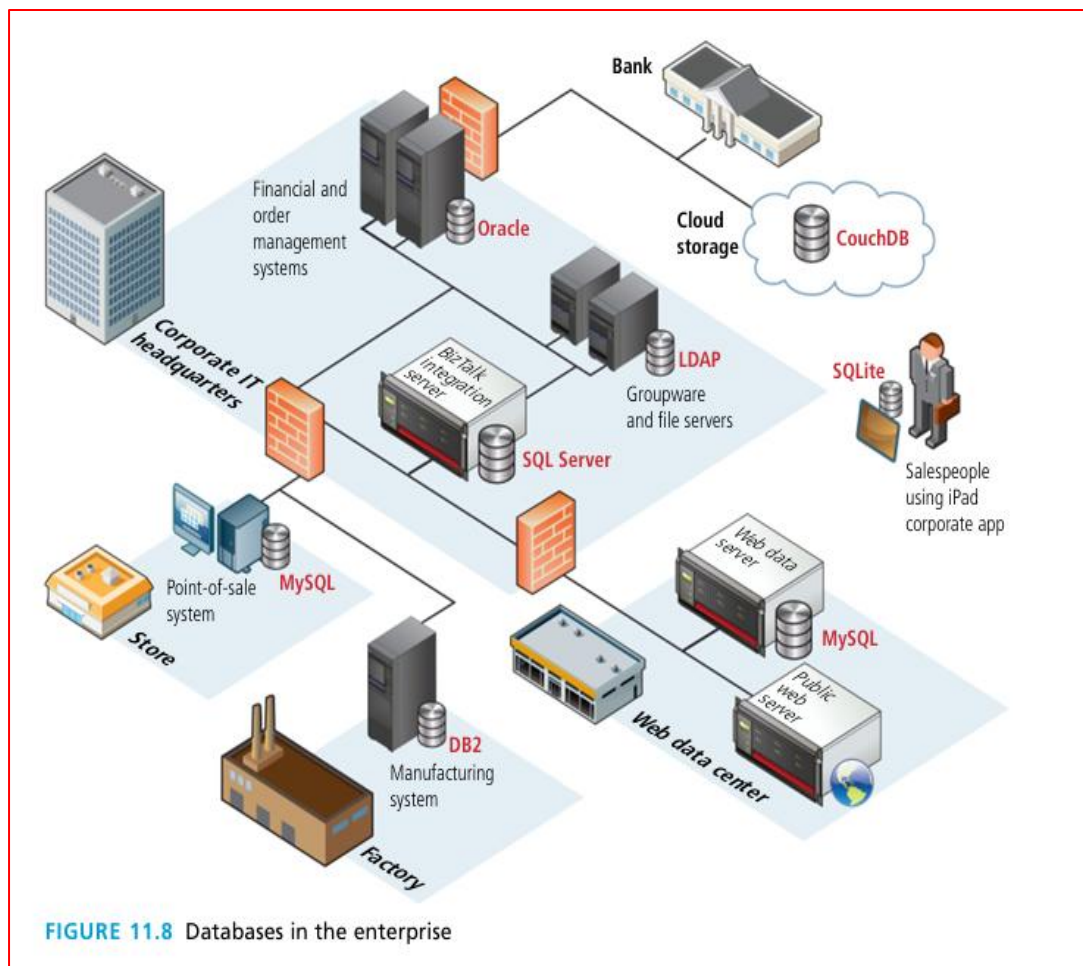


Note that in this example, the two foreign keys in the intermediate table are combined to create a composite key. Alternatively, the intermediate could contain a separate primary key field.

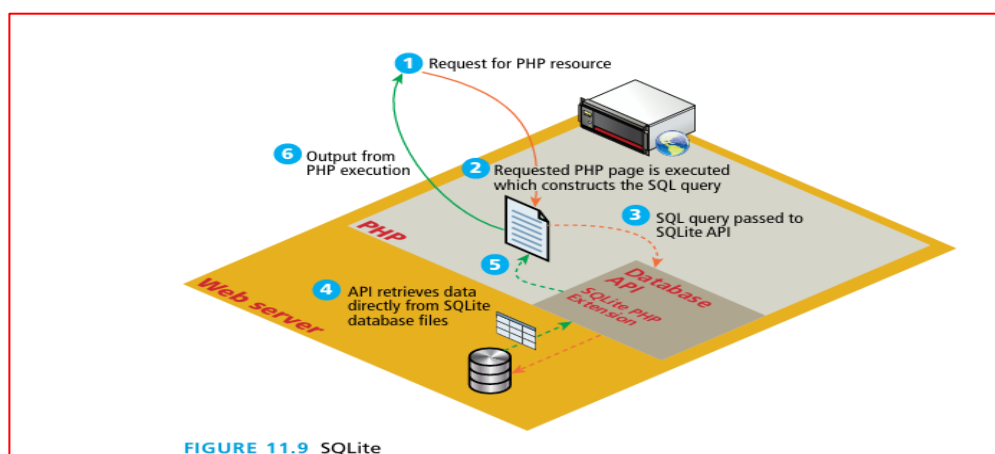
5.1.3 Database Options

Before we move on to the use of databases in MySQL, we should reiterate that there are a number of alternate database solutions. We earlier mentioned a variety of proprietary commerce enterprise database management systems such as Oracle Database, IBM DB2, and Microsoft SQL Server. These systems tend to be quite expensive, but provide a level of performance, features, and support that can be attractive for large-scale sites, especially if there were already legacy databases in use by the organization that either predate its web presence or are connected to a software system outside of the website, as shown in Figure 11.8.

It should be mentioned that although MySQL is free, it can be and is used for large and busy websites. Indeed many of the largest sites on the web, such as Facebook and Flickr, make use of some form of MySQL.



While MySQL is exceptionally popular as a web database, there are other open-source database systems. Perhaps the most common of these is PostgreSQL, which is a sophisticated object-relational DBMS. With the spread of memory constrained mobile devices, many developers have become interested in smaller database systems with fewer features. Perhaps the most widely used of these is SQLite, a software library that typically is integrated directly within an application rather than running as a separate process like most database management systems, as shown in Figure 11.9. One advantage of the SQLite approach for web developers is that no additional database software is required on the web server, which can be very attractive in hosting environments that charge for database server connectivity.



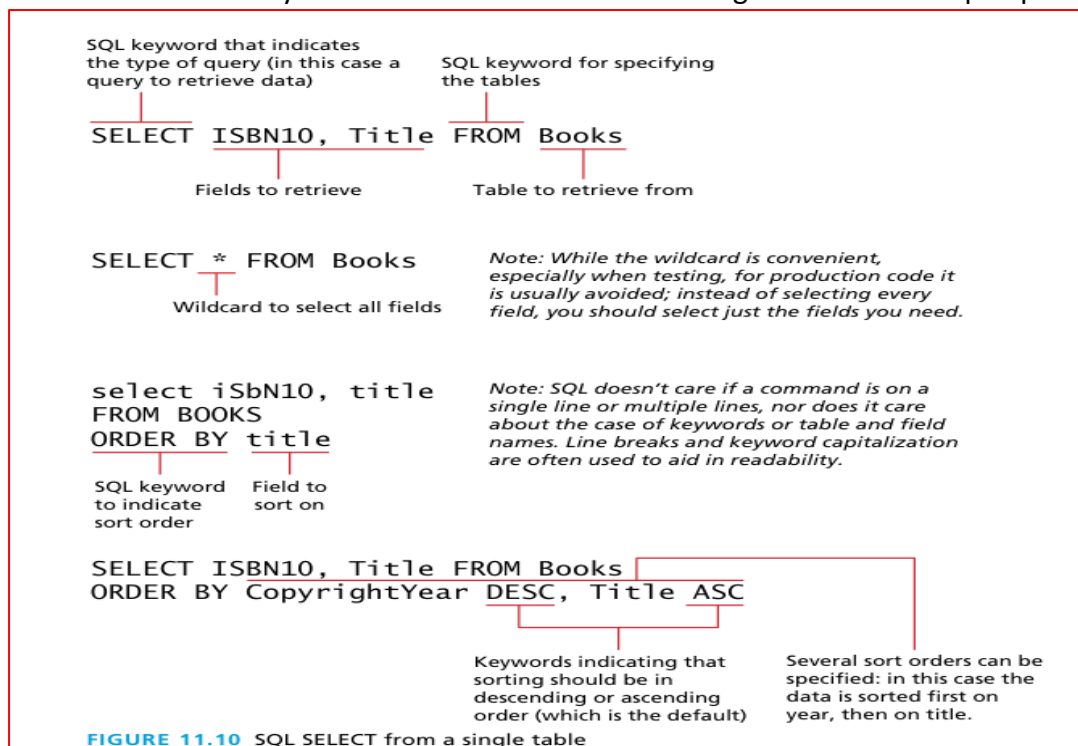
Finally, there is another category of database that is gaining some headway in the web world: the so-called no-SQL database. These databases do not make use of SQL, are not relational in how they store data, and are optimised to retrieve data using simple key-value syntax similar to that used with associative arrays in PHP. These types of databases are useful for information that is not stored relationally but within documents or for very large data sets. Some examples of document oriented no-SQL database systems include CouchDB and mongoDB. Other types of no-SQL databases are those that are optimised for storing (and retrieving) gigantic quantities of data, such as web server logs, geographical data, and information such as Twitter posts. Some examples of no-SQL database systems optimised for working with large data sets include the open-source Hypertable and Cassandra and the proprietary Amazon SimpleDB. Many of Google's services use its own proprietary no-SQL database called BigTable.

5.2 SQL

Relational databases almost universally use Structured Query Language or, as it is more commonly called, SQL (pronounced sequel) as the mechanism for storing and manipulating data. While each DBMS typically adds its own extensions to SQL, the basic syntax for retrieving and modifying data is standardized and similar. It will provide examples of some of the more common SQL commands.

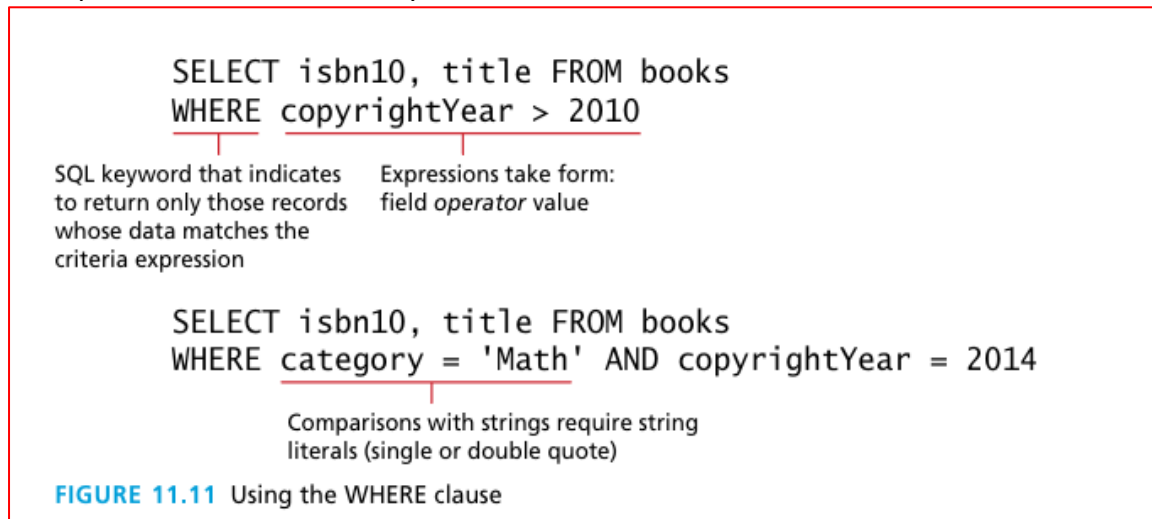
5.2.1 SELECT statement

- The SELECT statement is by far the most common SQL statement. It is used to retrieve data from the database. The term query is sometimes used as a synonym for running a SELECT statement (though "query" is used by others for any type of SQL statement).
- The result of a SELECT statement is a block of data typically called a result set. Figure 11.10 illustrates the syntax of the SELECT statement along with some example queries.

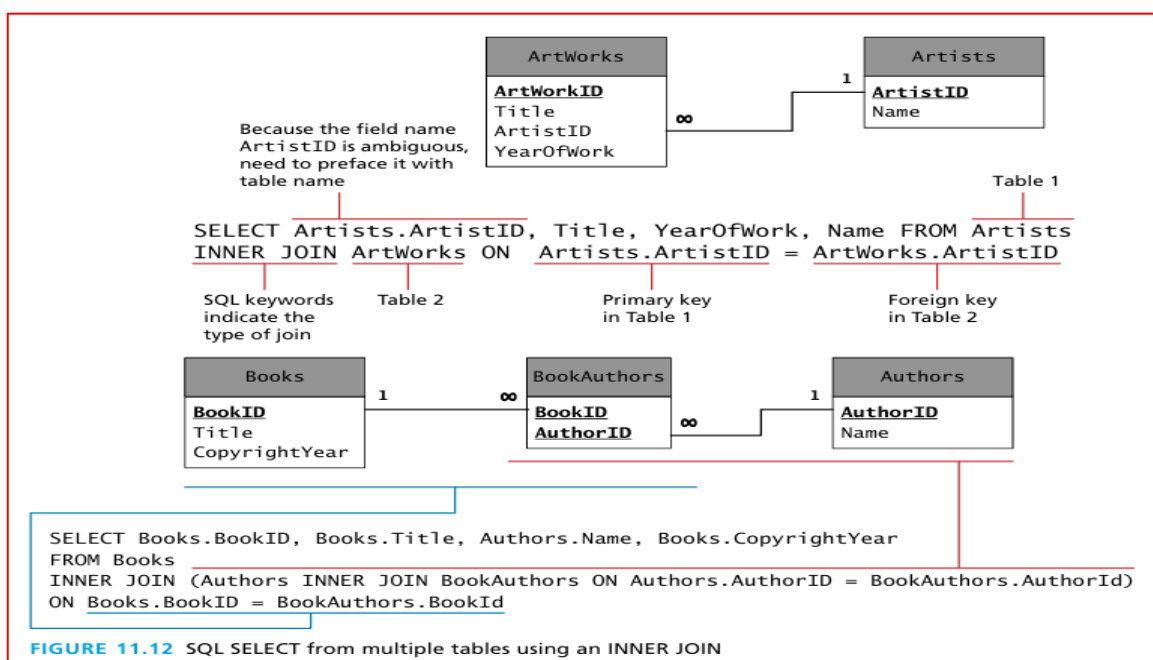


The examples in Figure 11.10 return all the records in the specified table. Often we are not interested in retrieving all the records in a table but only a subset of the records. This is accomplished via the WHERE clause, which can be added to any SELECT statement.

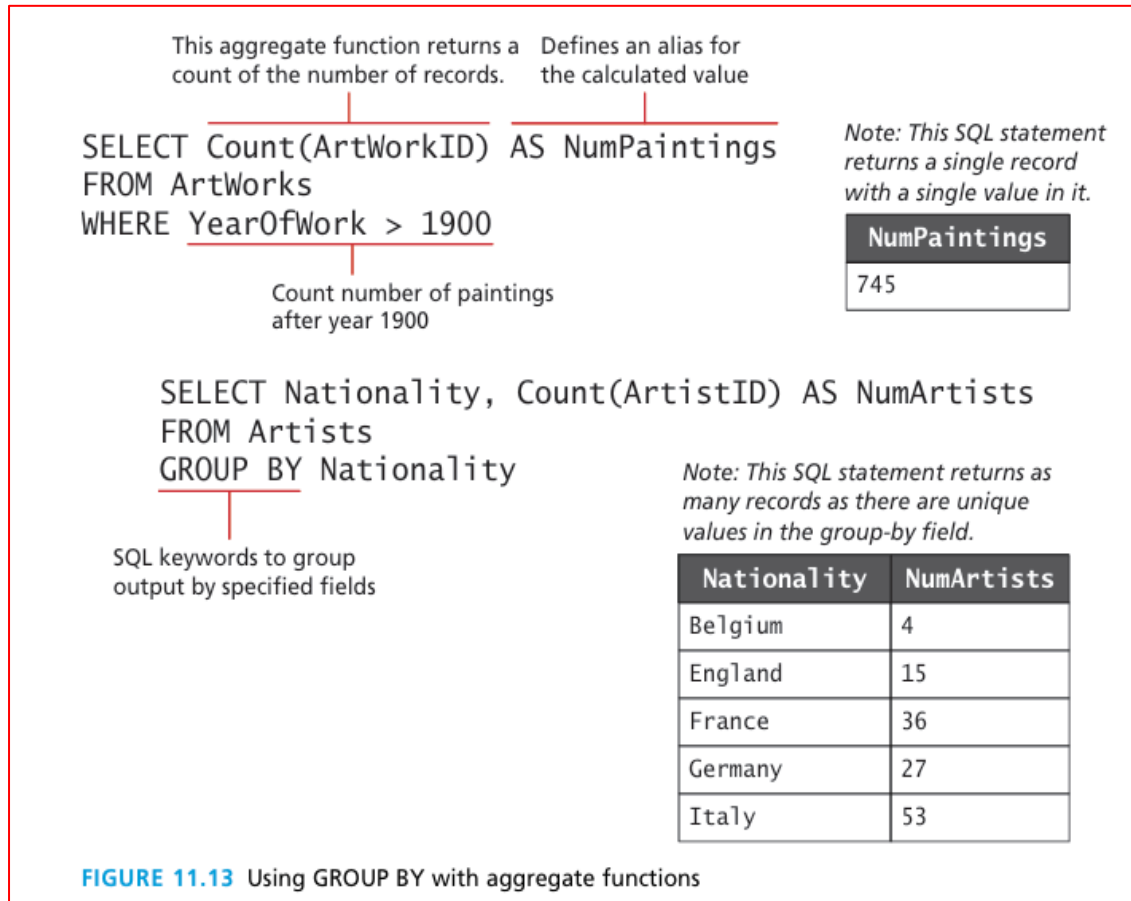
- That is, the WHERE keyword is used to supply a comparison expression that the data must match in order for a record to be included in the result set. Figure 11.11 illustrates some example uses of the WHERE keyword.



- The examples in Figures 11.10 and 11.11 retrieve data from a single table. Retrieving data from multiple tables is more complex and requires the use of a join. While there are a number of different types of join, each with different result sets, the most common type of join (and the one we will be using in this book) is the inner join. When two tables are joined via an inner join, records are returned if there is matching data (typically from a primary key in one table and a foreign key in the other) in both tables. Figure 11.12 illustrates the use of the INNER JOIN keywords to retrieve data from multiple tables.

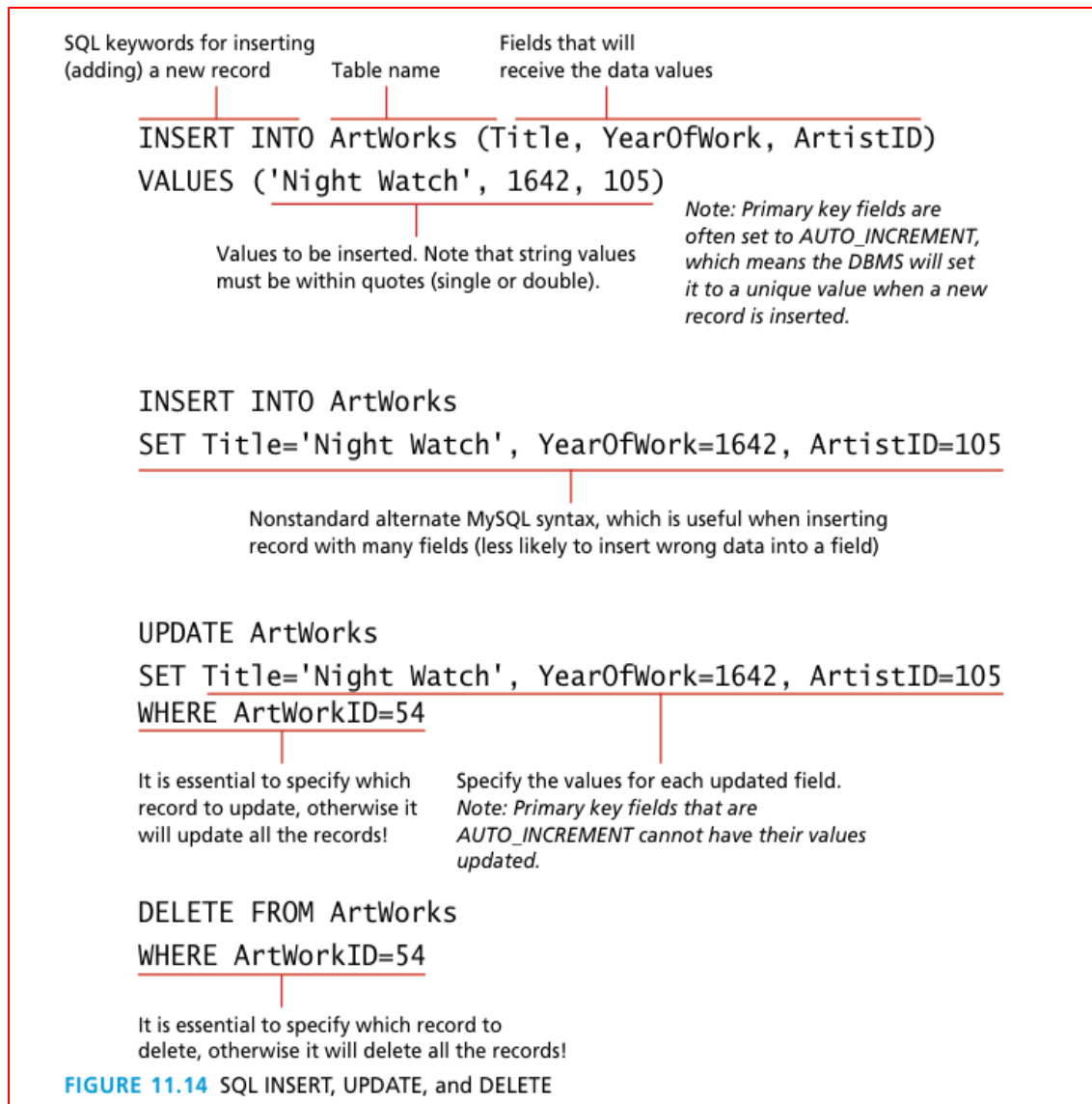


- Finally, you may find occasions when you don't want every record in your table but instead want to perform some type of calculation on multiple records and then return the results. This requires using one or more aggregate functions such as SUM() or COUNT(); these are often used in conjunction with the GROUP BY keywords. Figure 11.13 illustrates some examples of aggregate functions and a GROUP BY query.



5.2.2 INSERT, UPDATE, and DELETE statements

The INSERT, UPDATE, and DELETE statements are used to add new records, update existing records, and delete existing records. Figure 11.14 illustrates the syntax and some examples of these statements.



5.2.3 Transactions

A transaction refers to a sequence of steps that are treated as a single unit, and provide a way to gracefully handle errors and keep your data properly consistent when errors do occur.

Local transactions:

MySQL (and other enterprise quality DBMSs) supports local transactions through SQL statements or through API calls.

The SQL for transactions use the **START TRANSACTION**, **COMMIT**, and **ROLLBACK** commands.

For instance, the SQL to update multiple records with transaction support would look like that shown in Listing 11.1.

```

/* By starting the transaction, all database modifications within
the transaction will only be permanently saved in the database
if they all work */

START TRANSACTION

INSERT INTO orders ...
INSERT INTO orderDetails ...
UPDATE inventory ...

/* if we have made it here everything has worked so commit changes */
COMMIT

/* if we replace COMMIT with ROLLBACK then the three database
changes would be "undone" */

```

LISTING 11.1 SQL commands for transaction processing

5.2.4 Data Definition statements

All of the SQL examples that you will use in this book are examples of the Data Manipulation Language features of SQL, that is, SELECT, UPDATE, INSERT, and DELETE. There is also a Data Definition Language (DDL) in SQL, which is used for creating tables, modifying the structure of a table, deleting tables, and creating and deleting databases.

```

CREATE TABLE table_name(
    column1 datatype,
    column2 datatype,
    column3 datatype,
    .....
    columnN datatype,
    PRIMARY KEY( one or more columns )
);

```

Example:

```

CREATE TABLE CUSTOMERS(
    ID      INT NOT NULL,
    NAME    VARCHAR (20) NOT NULL,
    AGE     INT NOT NULL,
    ADDRESS CHAR (25),
    SALARY  DECIMAL (18, 2),
    PRIMARY KEY (ID)
)

```

);

5.3 Database APIs

The term API stands for application programming interface and in general refers to the classes, methods, functions, and variables that your application uses to perform some task. Some database APIs work only with a specific type of database; others are cross-platform and can work with multiple databases.

5.3.1 PHP MySQL APIs

There are two basic styles of database APIs available in PHP. The first of these styles is a procedural API, which uses function calls to work with the database. The other style is an object-oriented API, which requires instantiating objects and invoking methods and properties.

There are three main database API options available in PHP when connecting to a MySQL database:

- **MySQL extension.** This was the original extension to PHP for working with MySQL and has been replaced with the newer `mysqli` extension. This procedural API should now only be used with versions of MySQL older than 4.1.3. (At the time of writing, the current version of MySQL was 5.7.3.)
- **`mysqli` extension.** The MySQL Improved extension takes advantage of features of versions of MySQL after 4.1.3. This extension provides both a procedural and an object-oriented approach. This extension also supports most of the latest features of MySQL.
- **PHP data objects (PDOs).** This object-oriented API has been available since PHP 5.1 and provides an abstraction layer (i.e., a set of classes that hide the implementation details for some set of functionality) that with the appropriate drivers can be used with any database, and not just MySQL databases. However, it is not able to make use of all the latest features of MySQL.

5.4 Accessing MySQL in PHP

The PHP for accessing databases will begin by showing you the techniques using the procedural `mysqli` extension.

the basic database connection algorithm is the same:

1. Connect to the database.
2. Handle connection errors.
3. Execute the SQL query.

4. Process the results.
5. Free resources and close connection.

Figure 11.20 illustrates these steps within a sample. The following sections will examine each of these steps in more detail.

```

<?php

try {
    $connString = "mysql:host=localhost;dbname=bookcrm";
    $user = "testuser";
    $pass = "mypassword";

    $pdo = new PDO($connString,$user,$pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "select * from Categories order by CategoryName";
    $result = $pdo->query($sql);

    while ($row = $result->fetch()) {
        echo $row['ID'] . " - " . $row['CategoryName'] . "<br/>";
    }
    $pdo = null;
}
catch (PDOException $e) {
    die( $e->getMessage() );
}

?>

```

FIGURE 11.20 Basic database connection algorithm

5.4.1 Connecting to a Database:

Before we can start running queries, our program needs to set up a connection to the relevant database. In the context of database programming, a connection is like a pipeline of sorts that allows communication between a DBMS and an application program. With MySQL databases, we have to supply the following information when making a database connection: the host or URL of the database server, the database name, and the database user name and password.

Listings 11.3 illustrate how to make a connection to a database using the mysqli.

```
// modify these variables for your installation
$host = "localhost";
$database = "bookcrm";
$user = "testuser";
$pass = "mypassword";

$connection = mysqli_connect($host, $user, $pass, $database);
```

LISTING 11.3 Connecting to a database with mysqli (procedural)

5.4.2 handling Connection errors

Unfortunately, not every database connection always works. Sometimes errors occur when trying to create a connection for the first time; other times connection errors occur with normally trouble-free code because there is a problem with the database server. Whatever the reason, we always need to be able to handle potential connection errors in our code.

```
$connection = mysqli_connect(DBHOST, DBUSER, DBPASS, DBNAME);

// mysqli_connect_error returns string description of the last
// connect error
$error = mysqli_connect_error();
if ($error != null) {
    $output = "<p>Unable to connect to database<p>" . $error;
    // Outputs a message and terminates the current script
    exit($output);
}
```

LISTING 11.7 Handling connection errors with mysqli (version 1)

```
$connection = mysqli_connect(DBHOST, DBUSER, DBPASS, DBNAME);

// mysqli_connect_errno returns the last error code
if ( mysqli_connect_errno() ) {
    die( mysqli_connect_error() ); // die() is equivalent to exit()
}
```

LISTING 11.8 Handling connection errors with mysqli (version 2)

5.4.3 Executing the Query

- How to retrieve the data from backend to front end.

If the connection to the database is successfully created, then you are ready to construct and execute the query. This typically involves creating a string that contains the SQL statement and then calling one of the query functions/methods as shown in Listings 11.11. Remember that SQL is case insensitive, so the use of uppercase for the SQL reserved words is purely a coding convention to increase readability.

```
$sql = "SELECT * FROM Categories ORDER BY CategoryName";  
  
// returns a mysqli_result object  
$result = mysqli_query($connection, $sql);
```

LISTING 11.11 Executing a SELECT query (mysqli)

So, what type of data is returned by these query functions? Although the comments in the listings indicate that different data types are returned, essentially both return a result set, which is a type of cursor or pointer to the returned data.

- How to send the data to front end to back end (server) or update the database or delete the data from database.

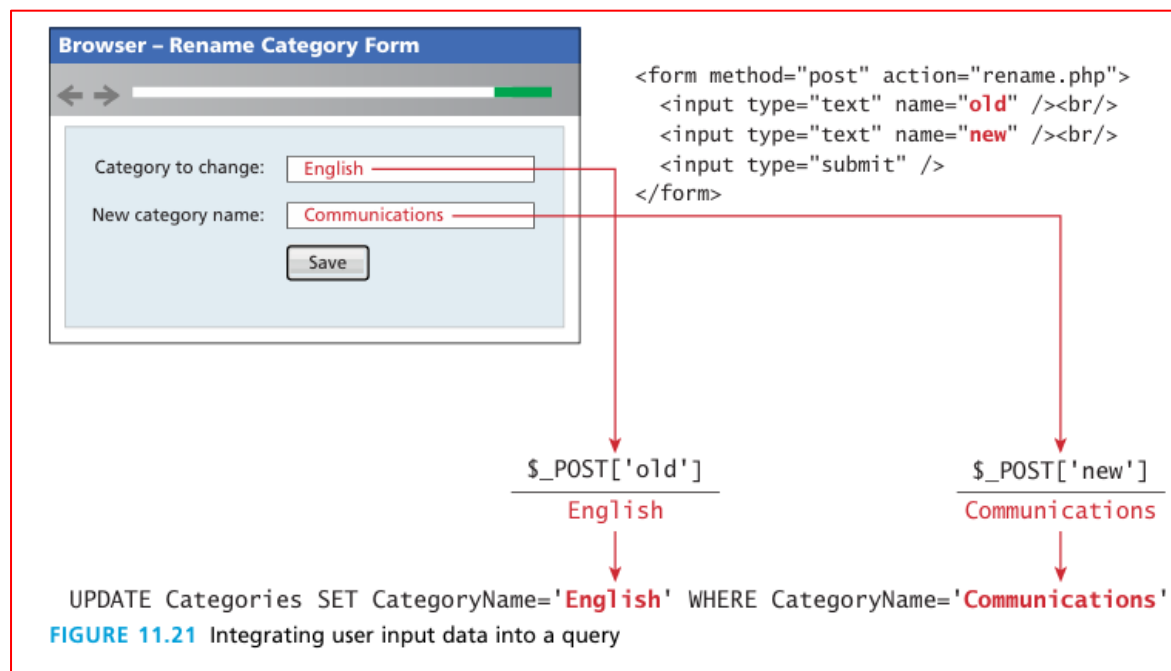
You may recall that not all SQL statements return data. INSERT, UPDATE, and DELETE statements instead perform an action on the data. Listings 11.13 illustrate an example update query.

```
$sql = "UPDATE Categories SET CategoryName='Web' WHERE  
      CategoryName='Business'";  
  
if ( mysqli_query($connection, $sql) ) {  
    $count = mysqli_affected_rows($connection);  
    echo "<p>Updated " . $count . " rows</p>";  
}
```

LISTING 11.13 Executing a query that doesn't return data (mysqli)

Integrating User Data

One of the most common database scenarios is that you have to run a query that uses some type of user input contained within a query string parameter, as shown in Figure 11.21.



prepared statements

The most important (and best) technique is to use prepared statements. A prepared statement is actually a way to improve performance for queries that need to be executed multiple times. When MySQL creates a prepared statement, it does something akin to a compiler in that it optimizes it so that it has superior performance for multiple requests. It also integrates sanitization into each user input automatically, thereby protecting us from SQL injection.

Listing 11.17 illustrates the code for using prepared statements using the procedural mysql approach.

```
// retrieve parameter value from query string
$id = $_GET['id'];

// construct parameterized query - notice the ? parameter
$sql = "SELECT Title, CopyrightYear FROM Books WHERE ID=?";

// create a prepared statement
if ($statement = mysqli_prepare($connection, $sql)) {
    // Bind parameters s - string, b - blob, i - int, etc
    mysqli_stmt_bindm($statement, 'i', $id);

    // execute query
    mysqli_stmt_execute($statement);

    // learn in next section how to access the returned data
    ...
}
```

LISTING 11.17 Using a prepared statement (mysqli)

Note: The most important thing to notice is the parameter symbol (the question mark) in the SQL string. This defines a placeholder for data that will be filled later when we bind the actual

data into the placeholder via the `mysqli_stmt_bindm()` function. This function then performs the appropriate sanitization of the user data, thereby providing some protection against injection attacks.

Example 2:

```
/* technique 1 - question mark placeholders */
$sql = "INSERT INTO books (ISBN10, Title, CopyrightYear, ImprintId,
    ProductionStatusId, TrimSize, Description) VALUES
    (?, ?, ?, ?, ?, ?, ?)";
$stmt = $pdo->prepare($sql);
$stmt->bindValue(1, $_POST['isbn']);
$stmt->bindValue(2, $_POST['title']);
$stmt->bindValue(3, $_POST['year']);
$stmt->bindValue(4, $_POST['imprint']);
$stmt->bindValue(4, $_POST['status']);
$stmt->bindValue(6, $_POST['size']);
$stmt->bindValue(7, $_POST['desc']);
$stmt->execute();

/* technique 2 - named parameters */
$sql = "INSERT INTO books (ISBN10, Title, CopyrightYear, ImprintId,
    ProductionStatusId, TrimSize, Description) VALUES (:isbn,
    :title, :year, :imprint, :status, :size, :desc) ";
$stmt = $pdo->prepare($sql);
$stmt->bindValue(':isbn', $_POST['isbn']);
$stmt->bindValue(':title', $_POST['title']);
$stmt->bindValue(':year', $_POST['year']);
$stmt->bindValue(':imprint', $_POST['imprint']);
$stmt->bindValue(':status', $_POST['status']);
$stmt->bindValue(':size', $_POST['size']);
$stmt->bindValue(':desc', $_POST['desc']);
$stmt->execute();
```

5.4.4 processing the Query results

If you are running a SELECT query, then you will want to do something with the retrieved result set, either display it, or perform calculations on it, or search for something in it, or some other operation. The technique for doing this with mysqli varies somewhat if one is using prepared statements. Listing 11.20 illustrates one technique for displaying content from a result set.

```

$sql = "SELECT * FROM Categories ORDER BY CategoryName";
// run the query
if ($result = mysqli_query($connection, $sql)) {
    // fetch a record from result set into an associative array
    while($row = mysqli_fetch_assoc($result))
    {
        // the keys match the field names from the table
        echo $row['ID'] . " - " . $row['CategoryName'] ;
        echo "<br/>";
    }
}

```

LISTING 11.20 Looping through the result set (mysqli—not prepared statements)

Notice that some type of fetch function must be called to move the data from the database result set to a regular PHP array. Once in the array, then you can use any PHP array manipulation technique. Figure 11.22 illustrates the process of fetching from the result set.

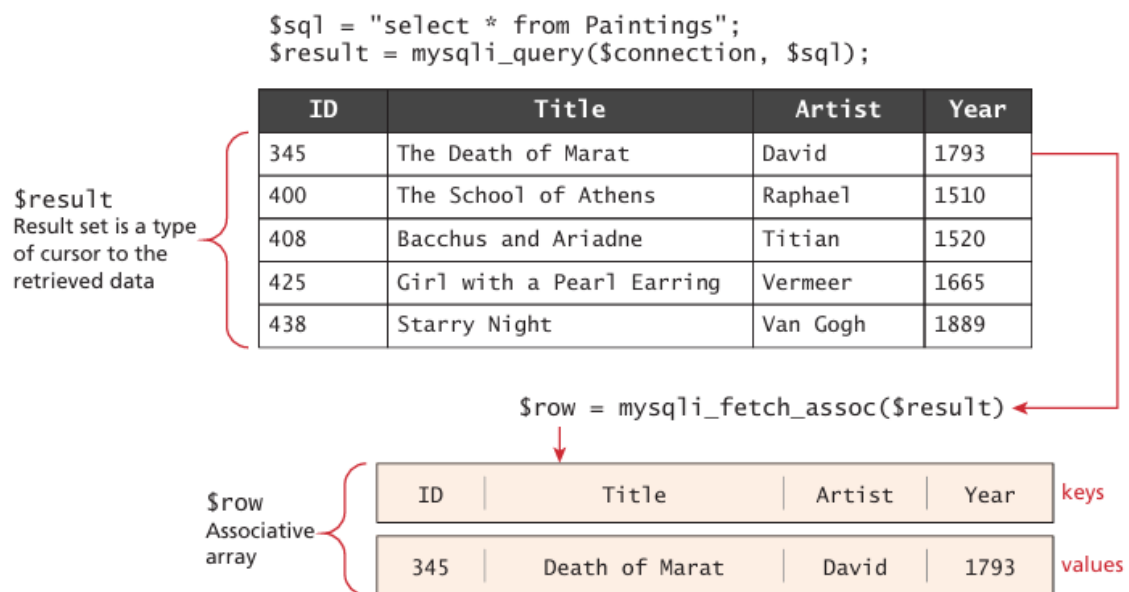


FIGURE 11.22 Fetching from a result set

The mysqli extension provides several fetch functions, which are listed in Table 11.2.

Type	Description
<code>mysqli_fetch_all()</code>	Fetches all result rows as an associative array, a numeric array, or both.
<code>mysqli_fetch_array()</code>	Fetches a result row as an associative array, a numeric array, or both.
<code>mysqli_fetch_assoc()</code>	Fetches a result row as an associative array.
<code>mysqli_fetch_field()</code>	Returns the next field in the result set. That is, it returns definition information about a single table column (not its data).
<code>mysqli_fetch_fields()</code>	Returns an array of objects representing the fields in a result set.
<code>mysqli_fetch_object()</code>	Returns the current row of a result set as an object.
<code>mysqli_fetch_row()</code>	Gets a result row as an numeric array.

TABLE 11.2 Fetch Functions

5.4.5 Freeing resources and Closing Connection

When you are finished retrieving and displaying your requested data, you should release the memory used by any result sets and then close the connection so that the database system can allocate it to another process. Listing 11.25 illustrates the code for closing the connection in `mysqli`.

```
// mysqli approach
$connection = mysqli_connect($host, $user, $pass, $database);
...
// release the memory used by the result set. This is necessary if
// you are going to run another query on this connection
mysqli_free_result($result);
...
// close the database connection
mysqli_close($connection);
```